



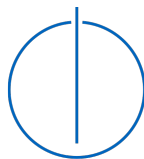
TUM SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Adaptivity between ACID and BASE in Concurrent Updates

B.Sc. Mykhailo Kazymyr





TUM SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Adaptivity between ACID and BASE in Concurrent Updates

Adaptivität zwischen ACID und BASE bei nebenläufigen Aktualisierungen

Author:	B.Sc. Mykhailo Kazymyr
Supervisor:	Prof. Dr.-Ing. Maribel Acosta
Advisor:	M.Sc. Zenon Zacouris
Submission Date:	01.09.2026



I confirm that this bachelor's thesis in informatics is my own work and I have documented all sources and material used.

Munich, 01.09.2026

B.Sc. Mykhailo Kazymyr

Acknowledgments

Abstract

The abstract will be written last, in about 200 words, and will follow the structure below.

Multi-model engineering systems keep several representations of the same product in agreement through automated consistency-preserving updates. Scheduling that work inside the transaction (ACID) keeps the models consistent at a high throughput cost, while deferring it to background reconcilers (BASE) is fast but leaves the models inconsistent by an amount the operator cannot choose. This thesis adds a hybrid execution mode that decides between the two per transaction, at admission time, and an adaptive oracle that makes that decision from the measured dirty fraction. The abstract will state the correctness problem the mixture creates and the arbiter that resolves it, then give the two headline measurements: the configured threshold acts as a bound on the inconsistency tail to within 1.3 percentage points across a tenfold range of the threshold and a fifteenfold range of reconciler provisioning, and this bound comes from the feedback loop rather than from mixing the two modes, since a fixed random split needs recalibration for every deployment. It will close on the honest summary: the adaptive oracle does not make the ACID/BASE trade-off cheaper, it makes it addressable.

Kurzfassung

Die deutsche Kurzfassung wird eine Übersetzung des englischen Abstracts und erst am Ende geschrieben.

Contents

Acknowledgments	iii
Abstract	iv
Kurzfassung	v
1. Introduction	1
1.1. Motivation	1
1.2. Problem statement	1
1.3. Formal problem definition	2
1.3.1. System model	2
1.3.2. Execution modes and inconsistency	2
1.3.3. Mode oracles and the resulting problem	2
1.4. Research questions	3
1.5. Overview of the approach	3
1.6. Contributions	4
1.7. Structure of the thesis	4
2. Preliminaries	5
2.1. Transactions and concurrency control	5
2.1.1. Transactions and serialisability	5
2.1.2. ACID and two-phase locking	5
2.1.3. BASE and eventual consistency	5
2.2. Multi-model systems and consistency preservation	6
2.2.1. Models, semantic overlap and CPRs	6
2.2.2. The dirty set as a control signal	6
2.3. The base simulator	7
2.3.1. Execution engine and scheduling	7
2.3.2. Parameters and metrics	7
2.4. Notation	7
3. Related Work	9
3.1. Combining ACID and BASE	9
3.1.1. Static classification of transactions	9
3.1.2. Runtime-adaptive consistency	9
3.1.3. Bounded inconsistency	9
3.2. Multi-model consistency preservation	10

3.3. Simulating transactional multi-model systems	10
3.4. Discussion and delimitation	11
4. Approach	12
4.1. Starting point and contribution boundary	12
4.2. Why a hybrid mode	12
4.3. The mixing hazard	13
4.4. The locked-XOR-dirty arbiter	13
4.4.1. The rule	13
4.4.2. Deadlock freedom	13
4.4.3. Blocking cost	13
4.5. Dispatcher architecture	14
4.6. Mode oracles	15
4.6.1. The oracle interface	15
4.6.2. The random oracle	15
4.6.3. The adaptive oracle	15
4.6.4. Control parameters	16
4.6.5. Signal sampling	16
4.7. Implementation notes	16
5. Experiments	17
5.1. Experimental settings	18
5.1.1. Simulator baseline configuration	18
5.1.2. Implementation and execution	19
5.1.3. Metrics	19
5.2. X0: Arbiter validation	21
5.3. X1: Reconciler-processor sweep	22
5.3.1. Validation against the reference results	24
5.3.2. Where the oracle engages	24
5.3.3. Is the hybrid a weighted average of its two modes?	27
5.4. X2: Consistency-budget sweep	28
5.4.1. The critical threshold as a control knob	29
5.4.2. Findings	31
5.4.3. The budget shifts work out of ACID, it does not make ACID cheaper	33
5.5. X3: Removing the feedback signal	34
5.5.1. Results of the probability sweep	35
5.5.2. Throughput at matched inconsistency	37
5.5.3. Findings	40
5.6. X4: Semantic overlap degree sweep	41
5.6.1. Reading the sweep	41
5.6.2. Findings	44
5.7. X5: Resources per transaction sweep	44
5.7.1. Reading the sweep	46

5.7.2. Findings	47
5.8. Ablation: the smoothing factor and its removal	47
5.8.1. The $\alpha = 0$ anchor	48
5.8.2. The bound does not depend on the smoothing	49
5.8.3. What the smoothing changed	49
5.8.4. Why the smoothing was removed	50
5.8.5. Confirmed by re-running the sweeps	51
5.9. The effective recovery threshold	51
5.10. Threats to validity	54
5.11. The earlier filtered runs	56
6. Conclusion and Future Work	58
6.1. Summary	58
6.1.1. RQ1: making the hybrid mode well defined	58
6.1.2. RQ2: performance and tunability	58
6.1.3. RQ3: is the feedback necessary?	58
6.2. Lessons learned	59
6.3. Future work	60
A. Additional material	62
A.1. Full parameter table	62
A.2. Per-seed result tables	62
A.3. Full grids	62
A.4. Implementation defects found and fixed	62
A.5. Supplemental material and AI usage disclosure	62
List of Figures	63
List of Tables	64
Bibliography	66

1. Introduction

1.1. Motivation

Engineering systems maintain several models of the same product side by side: CAD geometry, a simulation mesh, a written specification. These models overlap semantically, in the sense that the same underlying fact is represented in more than one of them. Keeping the representations in agreement is automated through Consistency Preservation Rules (CPRs), which take an update made in one model and propagate it into every model that shares the affected information.

Three overlapping models — CAD geometry, simulation mesh, specification — with a write landing in one of them and the induced CPR work drawn twice: once on the critical path (ACID) and once deferred to a background reconciler (BASE), with the transiently inconsistent resources marked in the second case.

Figure 1.1.: Motivating example: an update to one model propagates to two others through consistency preservation rules, either inside the transaction or after it commits.

The open question is *when* that propagation is scheduled. Under ACID semantics the CPR work sits on the critical path: a transaction does not commit until every consequence of its write has been applied, so the models are never observably inconsistent, and throughput pays for it. Under BASE semantics the CPR work is handed to background reconcilers. Foreground throughput is high, but the models are transiently inconsistent, and how inconsistent they get is not something the operator chooses.

Neither mode offers a way to state how much inconsistency is acceptable. That is the gap this thesis addresses.

1.2. Problem statement

Under pure BASE, the inconsistency level is an emergent property of the configuration. The only lever available is the number of reconciler processors, R . It is coarse, indirect, and it costs foreground throughput across the whole system. Under pure ACID, inconsistency is zero by construction and throughput collapses. In neither case can an operator ask for a given consistency level and have the system deliver it.

This thesis introduces a HYBRID execution mode, in which the choice between ACID and BASE is made per transaction, at admission time, by a component called a *mode oracle*. The adaptive oracle makes that choice from a threshold on the measured dirty fraction. The claim under test is that this turns inconsistency from an emergent outcome into a configurable target.

1.3. Formal problem definition

1.3.1. System model

A multi-model system is a finite set of resources $\mathcal{X}$ with $|\mathcal{X}| = N$, partitioned across models $M_1, \dots, M_m$. Semantic overlap is a relation on $\mathcal{X}$ that holds between resources representing the same underlying fact. A consistency preservation rule maps a write on a resource x to the set $\text{cpr}(x) \subseteq \mathcal{X}$ of resources that must be updated in consequence; applying it may trigger further rules, up to a bounded CPR depth.

1.3.2. Execution modes and inconsistency

A root transaction t carries a mode $\mu(t) \in \{\text{ACID}, \text{BASE}\}$, fixed at admission and inherited by every bookkeeper transaction it spawns. Under ACID, the transitive CPR closure of t 's writes is applied inside t 's boundary and t commits only when that closure is complete. Under BASE, t commits after its own writes and the closure is handed to background reconcilers.

The dirty set $D_k \subseteq \mathcal{X}$ is the set of resources carrying a CPR update that has not yet been applied, and $d_k = |D_k|/N$ is the dirty fraction. A run therefore has an inconsistency profile $\{d_k\}$ over simulated time rather than a single inconsistency value. It is summarised by its 95th percentile $p_{95}(d)$: the level that the dirty fraction stays below for at least 95% of simulated time.

1.3.3. Mode oracles and the resulting problem

A mode oracle is a function Ω that produces $\mu(t)$ for each arriving root transaction from the state observable at admission. Pure ACID and pure BASE are the two constant oracles. An oracle is *feedback-driven* if its output depends on the measured dirty fraction and *fixed* if it does not; the random oracle, which returns ACID with probability p , is the fixed oracle used as a baseline.

Given a system instance, a reconciler provisioning R and a target inconsistency level c , the problem is to find an oracle Ω that maximises throughput subject to

$$p_{95}(d) \leq c + \varepsilon \tag{1.1}$$

for a small tolerance ε , using the same setting of its control parameter for every R . The two constant oracles solve degenerate versions of this problem. Pure ACID satisfies Equation 1.1 for every c with $d \equiv 0$, at the lowest throughput available; pure BASE maximises throughput

and satisfies it only when c happens to exceed the level that the configuration produces on its own. The region between them is unreachable without a per-transaction decision.

Allowing that decision creates a second, non-numerical problem. If two transactions in different modes run concurrently, a resource can be locked and dirty at the same time, a state that neither pure mode defines. The problem above is therefore an optimisation problem and a well-definedness problem at once, and the second must be solved first.

1.4. Research questions

RQ1 How can a multi-model transactional system integrate a *hybrid mode* that safely adapts between ACID and BASE semantics at runtime, when its subsystems hold overlapping resources?

RQ2 How does the hybrid mode compare with pure ACID and pure BASE execution in throughput and inconsistency, and can the inconsistency it permits be configured?

RQ3 Does choosing the mode from the system's own internal metrics outperform a random ACID/BASE assignment on the same throughput/inconsistency trade-off?

In the terms of Section 1.3, RQ1 asks whether a non-constant Ω can be made well defined at all, RQ2 asks what it costs and whether c controls the bound in Equation 1.1, and RQ3 asks whether Ω has to be feedback-driven or whether a fixed oracle suffices.

1.5. Overview of the approach

The approach has three parts. First, a HYBRID execution mode extends the base simulator so that each root transaction is tagged ACID or BASE at admission and executes accordingly, with both paths sharing the same resources, processors and CPR machinery. Second, because the two paths now coexist, an arbiter defines what happens when they meet on the same resource: ACID transactions wait for a dirty resource to be reconciled, and BASE transactions yield on a locked one. This single rule makes the mixed state well defined, and because the wait edge it adds points one way only, it introduces no new deadlock. Third, the tagging decision is delegated to a mode oracle. The adaptive oracle switches to ACID when the smoothed dirty fraction reaches the configured threshold c and back to BASE when it falls to a lower recovery threshold r ; the random oracle, which flips a coin with fixed probability p , is the baseline that isolates what the feedback contributes.

The evaluation follows the same logic. Five parameter sweeps, 640 simulation runs in total, establish where the oracle engages, what the mixture costs, whether c controls the realised inconsistency, and whether a fixed oracle can do the same job.

Block diagram of the request path — admission, mode oracle, the ACID and BASE dispatchers, the shared lock table and dirty set, the reconciler pool — with the oracle's input arrow drawn back from the dirty set.

Figure 1.2.: Overview of the approach: mode assignment at admission, the two execution paths, and the arbiter between them.

1.6. Contributions

- C1 (RQ1)** The HYBRID execution mode and the locked-XOR-dirty arbiter that makes it well defined, together with the argument that it admits no new deadlocks (Section 4.4).
- C2 (RQ1)** Two mode oracles, an adaptive one that reads the system's own state and a random one used as a baseline, together with their control parameters (Section 4.6).
- C3 (RQ2)** A measurement of where the adaptive oracle engages, and a quantified explanation of what mixing the two modes costs: ACID transactions admitted during an ACID excursion block behind background reconciliation for a mean of up to 687 simulated seconds (Sections 5.3 and 5.4).
- C4 (RQ2)** Evidence that the critical threshold c works as a configurable bound on the inconsistency tail: across c from 2% to 30% and R from 1 to 15, the measured p_{95} never differs from the configured c by more than 0.74 percentage points (Section 5.4).
- C5 (RQ3)** Evidence that this bound comes from the feedback loop and not from hybrid execution in general. Aiming at the same target with a fixed random split requires a control parameter that has to be recalibrated for every reconciler count, and whose realised effect varies by up to a factor of 44 across the range studied (Section 5.5).

1.7. Structure of the thesis

Chapter 2 introduces the concepts the thesis relies on: transactions and two-phase locking, BASE and reconciliation, multi-model systems and CPRs, the base simulator this work extends, and the two signal-handling ideas the oracle uses. Chapter 3 positions the work against the literature on combining ACID and BASE, on switching consistency at runtime, and on bounded inconsistency. Chapter 4 describes the approach: the hybrid mode, the correctness problem it creates, the arbiter that resolves it, and the two oracles. Chapter 5 gives the experimental settings and reports the four sweeps. Chapter 6 answers each research question, records the lessons learned, and lists the open questions.

2. Preliminaries

2.1. Transactions and concurrency control

2.1.1. Transactions and serialisability

A transaction is a partial order of read and write steps over resources. A schedule of concurrently executing transactions is correct if it is equivalent to some serial execution of the same transactions [1]. This is the only correctness criterion used in the thesis; the vocabulary is needed mainly to state what the ACID execution path guarantees and what the BASE path gives up.

2.1.2. ACID and two-phase locking

The simulator enforces isolation with conservative two-phase locking and cautious waiting [2]. A transaction declares the resources it needs before it begins, acquires them as a set rather than incrementally, and releases them only after its final step. A transaction that would have to wait for a resource held by another does not acquire a partial lock set and hold it while waiting.

Three consequences of this scheme are used later. Locks are never acquired after the first release, so no transaction can extend its lock set while holding one. A waiting transaction holds no locks that another transaction is waiting for. Therefore the wait-for relation among ACID transactions cannot contain a cycle, and pure ACID execution is deadlock-free by construction. Section 4.4.2 shows that the arbiter introduced in this thesis preserves that property.

2.1.3. BASE and eventual consistency

BASE trades immediate consistency for availability and throughput [3, 4]: a transaction commits before all of its consequences have been applied, and the system converges afterwards. Two terms recur throughout the thesis. A resource is *dirty* when a consistency-preserving update that concerns it has not yet been applied. *Reconciliation* is the background work that applies those pending updates and clears the resources again.

2.2. Multi-model systems and consistency preservation

2.2.1. Models, semantic overlap and CPRs

A model is a set of resources describing one view of the product [5]. Two models exhibit *semantic overlap* when they contain resources representing the same underlying fact; the overlap degree is a configuration parameter of the simulator. A consistency preservation rule (CPR) states that a write to one resource requires updates to the resources that overlap it. Rules compose: an update produced by a rule may itself trigger further rules, and the resulting chain is bounded by a maximum CPR depth.

The transactions that carry this induced work are called *bookkeeper* transactions. They are spawned by the root transaction whose write triggered the rule and inherit its execution mode. A rule also has a referential action: ON_CASCADE propagates the update, whereas ON_RESTRICT rejects the originating write instead. All experiments in this thesis use ON_CASCADE.

2.2.2. The dirty set as a control signal

Under BASE semantics, the resources awaiting a CPR update form the dirty set D_k , of size $|D_k|$, and the dirty fraction is $d_k = |D_k|/N$ over the N resources of the system. Reconciler processors drain this set in the background; their number R is the main provisioning parameter of the system. The dirty fraction is the quantity every result in this thesis is about: it is what the adaptive policy observes, what the critical threshold bounds, and what the reported percentiles summarise.

It is also the only signal available for deciding *when* the system should stop accumulating inconsistency, which makes it a control signal and not merely a metric. The simplest policy over such a signal is a single threshold: run BASE below it, ACID above it. A single threshold on a fluctuating signal causes flapping, that is, the system switches back and forth every time the signal crosses it, paying the cost of a mode change without staying in the new mode long enough to benefit from it. The standard remedy is a *hysteresis band*: separate thresholds for entering and for leaving a state. Here the critical threshold c governs entry into ACID and the lower recovery threshold r governs the return to BASE, so the interval $[r, c]$ is the margin the signal must traverse before a second switch can occur. Section 4.6.3 states the resulting rule for the oracle built in this thesis.

Two properties of the dirty fraction make this band cheap to reason about. It carries no measurement noise — d_k is an exact count of a set the simulator maintains, not an estimate — and it is bounded in $[0, 1]$, so both thresholds are directly interpretable as fractions of the resource population.

2.3. The base simulator

2.3.1. Execution engine and scheduling

The work extends the discrete-event simulator of a transactional server for multi-model systems introduced by Zacouris and Acosta [6, 7]. Time advances in fixed steps of one simulated second. Transactions are admitted, expanded into their CPR closure, scheduled onto IO and CPU processor pools, and either commit or abort; under BASE the closure is instead queued for the reconciler pool. Because the clock is virtual, results do not depend on the speed of the host machine.

2.3.2. Parameters and metrics

The simulator exposes the workload profile, the number and size of models, the semantic overlap degree, the processor and reconciler counts, the lock budget, and the maximum number of resources and models per transaction. The workload profile decides how root transactions arrive; the closed-loop profile EQUILIBRIUM used throughout this thesis holds the number of in-flight transactions constant and admits a new one whenever one completes, so the load is stationary by construction (Section 5.1.1). The simulator also generates its own workload from seeds, so no external dataset is involved, and it exports per-run metrics as CSV. The parameter table of [7] is reproduced here, and the configuration used in Chapter 5 is stated as a difference against it.

2.4. Notation

Table 2.1 fixes the symbols used throughout. Lowercase symbols are fractions of the resource population and uppercase symbols are absolute counts or configuration values. The index k counts successive samples, which are taken once per admission event rather than once per unit of simulated time — a property with consequences of its own (Section 5.9).

Table 2.1.: Notation used throughout this thesis.

Symbol	Meaning	Definition	Implementation
R	Reconciler processors	—	<code>numberOfReconcilerProcessors</code>
N	Total resources	—	<code>totalResources</code>
L	Max resources per txn	—	<code>maxTransactionResources</code>
D_k	Dirty resources (count)	—	<code>DirtySetManager.size()</code>
d_k	Dirty fraction	D_k/N	<code>dirtyFraction</code>
c	Critical threshold	—	<code>criticalDirtyFraction</code>
r	Recovery threshold	— ($0 \leq r \leq c$)	<code>recoveryDirtyFraction</code>
p	Random-oracle ACID prob.	—	<code>randomOracleAcidProbability</code>
ϕ_{time}	ACID time-share	(Section 5.1.3)	<code>acid_fraction_time</code>
ϕ_{comp}	ACID completion-share	(Section 5.1.3)	<code>acid_fraction_completed</code>

3. Related Work

3.1. Combining ACID and BASE

3.1.1. Static classification of transactions

Salt [8] is the closest neighbour of this work. It lets a developer convert the few performance-critical ACID transactions of an application into BASE transactions, split into smaller steps, while the remaining ACID transactions keep their guarantees through an additional isolation level. The mixture of the two semantics is therefore explicit and intended, as it is here.

Two differences separate it from this thesis. Salt decides *which* transactions are BASE, statically, when the application is written, whereas this thesis decides *when* to use each mode, at runtime, from measured system state. And Salt targets single-schema replicated storage, whereas the setting here is a multi-model system in which inconsistency is produced by automatic propagation between models.

RedBlue consistency [9] makes a similar classification at a finer granularity: operations labelled blue execute without coordination and operations labelled red require it, with the labelling fixed at design time and constrained by commutativity requirements. The same contrast applies, one level down.

3.1.2. Runtime-adaptive consistency

Consistency Rationing [10] is the closest methodological precedent. Data is divided into categories that receive different guarantees, and the middle category switches its consistency level at runtime according to a policy driven by monitored statistics and expected cost. The mode oracle of this thesis is the same idea applied to a different signal: the measured dirty fraction rather than an economic estimate.

Harmony [11] adapts the number of replicas consulted per read in a cloud storage system based on an estimate of the probability of returning stale data, which is again a runtime decision driven by a monitored proxy.

3.1.3. Bounded inconsistency

Yu and Vahdat [12] treat consistency as a continuous quantity and allow applications to bound the divergence they tolerate along several axes through conits. The critical dirty fraction used here is a conit-style numerical bound, and the tracking result of Section 5.4 is direct evidence that it behaves as one in this setting.

3.2. Multi-model consistency preservation

Vitruvius [5] and the line of work on consistency preservation rules provide the setting itself: automated propagation of updates between models that represent the same system from different viewpoints. That work treats propagation as a correctness mechanism to be specified and executed, rather than as a source of transient inconsistency to be measured and bounded.

3.3. Simulating transactional multi-model systems

This thesis extends an existing simulator, and the work behind it is the most directly relevant related work of all.

Transaction simulators have a long history in single-model settings. Ries and Stonebraker [13] used one to study locking granularity under a workload held in equilibrium, and Dandamudi and Au [14] extended that analysis to multiprocessor database systems. Both establish the method this thesis relies on: comparing concurrency-control policies under identical synthetic workloads, in virtual time, so that the comparison is not confounded by the host machine.

Zacouris and Acosta [6] carry that method into multi-model systems. Their simulator models a transactional server in which an update to one model triggers consistency-preserving updates in the models that overlap it, executed by bookkeeper transactions, and lets the user vary concurrency, transaction size, semantic overlap and CPR structure. The subsequent study [7] adds the second execution semantics and the comparison this thesis starts from: under ACID, the consistency-preserving work runs inside the triggering transaction, and throughput degrades sharply as semantic overlap and CPR density grow; under BASE, the work is deferred to background reconciliation, and foreground throughput becomes largely insensitive to both, at the price of transient inconsistency whose level depends on how many processors are allocated to reconciliation.

Two results of that work are the direct starting point here. The first is the throughput/consistency trade-off governed by reconciler provisioning: too few reconcilers leave inconsistency high, too many starve foreground execution. Section 5.3.1 reproduces its shape and uses it to validate the extended simulator. The second is the observation that the best allocation depends on the configuration of the multi-model system, which is precisely the property that makes reconciler provisioning an awkward control: it is a knob whose correct setting must be found again for every deployment.

What that work does not provide, and what this thesis adds, is a way to select the execution semantics *per transaction* and *at runtime*. In the base simulator, ACID and BASE are alternative configurations of a whole run; here they coexist within one run, which requires the arbiter of Section 4.4 and makes the tolerated inconsistency a configurable parameter rather than an outcome of the provisioning.

3.4. Discussion and delimitation

Table 3.1 states the differentiation on the dimensions that actually separate the work, rather than by topic.

Table 3.1.: Differentiation from related work.

Work	What is decided	When	Target system	Knob is stated in
Salt [8]	which txn is BASE	authoring time	single schema, replicated	code annotations
RedBlue [9]	which op is weak	authoring time	single schema, replicated	operation labels
Cons. Rationing [10]	consistency level	runtime	single schema, replicated	expected cost
Harmony [11]	replicas read	runtime	key-value store	staleness estimate
Yu & Vahdat [12]	divergence allowance	runtime	replicated state	conit bounds
Vitruvius [5]	— (always applied)	—	multi-model	—
Base simulator [6, 7]	semantics per run	configuration time	multi-model, CPR-driven	reconciler count
This thesis	mode per transaction	admission time	multi-model, CPR-driven	the controlled quantity

Two rows carry the delimitation. Against the base simulator, the difference is granularity and timing: semantics chosen per transaction at admission rather than per run at configuration time. Against everything above it, the difference is the last column, and it follows from the RQ3 result rather than from the literature. The existing runtime-adaptive systems are steered by a proxy — an expected cost, a staleness estimate, a reconciler count — so the consistency they achieve is emergent, whereas c is set in the units of the quantity it bounds.

Beyond the base simulator, all of the work above targets distributed replicated storage under a single schema, where inconsistency means divergence between copies of the same datum. None of it addresses inconsistency generated by automatic consistency-preserving updates between semantically overlapping models. That is the gap this thesis fills.

4. Approach

4.1. Starting point and contribution boundary

The base simulator described in Section 2.3 is received work [6, 7]. Table 4.1 separates it from what this thesis adds, so that the boundary is explicit before the design is described.

Table 4.1.: Received work against own contribution.

Received (base simulator)	Added in this thesis
Execution engine, fixed time step, processor pools	HYBRID execution mode and per-transaction mode tag
Lock table and conservative 2PL	Locked-XOR-dirty arbiter between the two modes
Dirty set and reconciler pool	Adaptive mode oracle (threshold, hysteresis)
CPR expansion and bookkeeper transactions	Random mode oracle with configurable p , as a baseline
Pure ACID and pure BASE runtimes	Metrics for mode switching, ACID share, blocking and excursions
Existing metric and CSV export	Four experiment runner scripts and their sweeps

One configuration value also differs from the reference setup, justified in Section 5.1.1: the run duration. Everything else stays at the reference configuration, which is what makes the validation in Section 5.3.1 meaningful.

4.2. Why a hybrid mode

The purpose of the hybrid mode is to turn inconsistency from an emergent outcome into a configurable budget. An operator sets a level; the system spends whatever ACID execution is needed to stay under it, and runs BASE otherwise.

The claim is deliberately restricted in one respect. The hybrid mode is not claimed to respond to changing load better than a static configuration. All experiments run under a stationary workload (EQUILIBRIUM), where there is no change to respond to and a well-tuned static configuration wins by construction. What the results support is configurability *across deployments*, not adaptation to load *within* one; the latter is untested here by construction and is listed as future work.

4.3. The mixing hazard

ACID's lock table and BASE's dirty set are independent mechanisms that never coexist in a pure run. Choosing the mode per transaction lets a resource be locked and dirty at the same time. The two states contradict each other: the lock says the value is being written under exclusive access, while dirtiness says an unapplied update is still pending on it. A mixed execution therefore has to define what happens when a transaction in one mode meets a resource marked by the other.

Two transactions and one resource. The ACID transaction holds a lock on it; the BASE transaction has left a pending CPR update on it. The state that neither pure mode defines is marked, with the two possible resolutions — wait or yield — as the edges out of it.

Figure 4.1.: The mixing hazard: with modes chosen per transaction, a resource can be held by an ACID lock and be in the dirty set at the same time.

4.4. The locked-XOR-dirty arbiter

4.4.1. The rule

The arbiter enforces a single invariant: no resource is simultaneously locked and dirty. It does so with one asymmetric rule. An ACID transaction that needs a dirty resource waits until reconciliation has cleared it. A BASE transaction that would dirty a locked resource yields instead, and its work is retried later. ACID waits on dirty; BASE yields on locked.

4.4.2. Deadlock freedom

Pure ACID execution is deadlock-free under conservative two-phase locking with cautious waiting (Section 2.1.2). The arbiter adds exactly one new kind of wait edge: from an ACID transaction to the reconciliation of a dirty resource. Reconcilers never wait on ACID transactions, because a BASE-side conflict is resolved by yielding rather than by blocking. The added edge therefore points one way only, no cycle that includes it can be formed, and the arbiter admits no deadlock that pure ACID did not already admit.

4.4.3. Blocking cost

The rule is safe but not free, and the cost falls entirely on the ACID side. A transaction touches up to $L = 50$ resources (`maxTransactionResources`). If those resources were drawn independently and uniformly at dirty fraction d , the blocking probability follows from asking the opposite question first: what is the chance that the transaction gets through *clean*, i.e. touches no dirty resource at all?

A single resource is clean with probability $1 - d$. With L resources drawn independently, the transaction is clean only if every one of them is clean, so the probabilities multiply:

$$P_{\text{clean}}(d) = \underbrace{(1 - d) \cdot (1 - d) \cdots (1 - d)}_{L \text{ resources}} = (1 - d)^L. \quad (4.1)$$

That is the whole meaning of $(1 - d)^L$: the probability that all L resources touched by the transaction happen to be clean. Blocking is the complementary event — at least one of the L resources is dirty — so its probability is one minus Equation 4.1:

$$P_{\text{block}}(d) = 1 - P_{\text{clean}}(d) = 1 - (1 - d)^L, \quad (4.2)$$

which for $L = 50$ gives 40% at $d = 0.01$, 64% at $d = 0.02$, 92% at $d = 0.05$ and 99.5% at $d = 0.10$. Blocking is thus not a rare event but the normal case for an ACID transaction admitted while the dirty set is large; across the X2 grid, blocked transactions number 0.59 to 1.12 per ACID admission (Table 5.7; that ratio is not a probability, see Section 5.10).

The measurements also show what this probability cannot explain. Since P_{block} saturates near one over most of the operating range, it cannot account for a throughput range spanning a factor of eight. What varies over that range — by a factor of 17, monotonically with R — is not *whether* an ACID transaction blocks but *for how long*. The mean blocking time at $c = 0.10$ falls from 392 s at $R = 1$ to 22.5 s at $R = 15$. At $R = 1$ the reconciler pool drains about 0.5 resources per second while a blocked transaction waits on a mean of 2.9 dirty resources, so foreground work serialises behind background work. It is the product of incidence and duration that produces the throughput valley of Section 5.3.2, and the same argument explains why raising c helps only sublinearly: a larger c lengthens each excursion, and hence each block, while reducing the number of ACID admissions.

4.5. Dispatcher architecture

Mode tagging is applied at admission, and a dispatcher per mode routes the tagged transaction to the corresponding execution path. Both paths draw on the same processor pools, the same lock table and the same CPR expansion; they differ in scheduling and bookkeeping, not in the work performed. That is what makes the comparison between configurations in Chapter 5 a comparison of policies rather than of implementations.

Internal structure of the server, with components shared by both modes drawn once and the two mode-specific dispatchers as the only fork. Shading separates received components from those added in this thesis, matching Table 4.1.

Figure 4.2.: Dispatcher architecture: one admission path, two dispatchers, shared processors, lock table, dirty set and reconciler pool.

4.6. Mode oracles

4.6.1. The oracle interface

A mode oracle assigns a mode at the admission of a root transaction. The tag is frozen for the lifetime of the transaction group: bookkeeper transactions inherit their root’s tag and never consult the oracle, and a transaction that aborts and retries keeps its original mode. Only two implementations exist, and they differ solely in what the decision is based on.

4.6.2. The random oracle

The random oracle returns ACID with a fixed probability p and BASE otherwise. It is not a strawman but the ablation baseline: the full hybrid machinery — the same arbiter, the same tagging, the same dispatchers — with exactly one component removed, the feedback signal. Without it, a good adaptive result would be equally consistent with “mixing the two modes at all helps” and would say nothing about the feedback loop. This is what makes RQ3 answerable.

One property of this baseline shapes the evaluation. The probability p fixes the share of transactions routed to ACID by construction ($\phi_{\text{comp}} = 0.4968$ to 0.5003 at $p = 0.5$, across all R), but it does not fix the share of *time* spent in ACID, which ranges from 0.52 to 0.79 over the same runs, because ACID transactions occupy more simulated time when reconciliation is slow. The routing share is controlled; the time share is emergent. This asymmetry makes the phrase “equal ACID budget” ambiguous, which is why X3 compares the two oracles on a measured outcome — the p_{95} dirty fraction — rather than on either ACID share.

4.6.3. The adaptive oracle

The adaptive oracle reads the live dirty fraction at the instant of the decision and switches between two states:

$$\text{BASE} \rightarrow \text{ACID} \quad \text{when} \quad d_k \geq c \quad (4.3)$$

$$\text{ACID} \rightarrow \text{BASE} \quad \text{when} \quad d_k \leq r \quad (4.4)$$

with the interval $[r, c]$ acting as the hysteresis band.

Two states, BASE and ACID, and two edges labelled with Equations 4.3 and 4.4. An inset plot of the dirty fraction over time, with c and r as horizontal lines and the band between them shaded, shows the anti-flap role of the band.

Figure 4.3.: The adaptive oracle as a two-state machine, with the entry and exit conditions on the edges.

Dirtiness is the oracle’s only decision signal, so every mode transition has a single unambiguous cause.

4.6.4. Control parameters

The oracle has two parameters: the critical threshold c and the recovery threshold r , which together define the hysteresis band. There is no third. An earlier version applied an exponentially weighted filter to the dirty signal, with a smoothing factor as a third parameter; it was removed after the ablation of Section 5.8 showed it left the bound untouched while lengthening every ACID excursion. Filtering earns its cost on a signal carrying measurement noise, and this one carries none: d_k is an exact count of a set the simulator maintains.

Because the comparison is against the live count, $r = 0$ needs no special handling — d_k reaches zero exactly. What remains is a granularity effect rather than an estimator lag: the rule is evaluated only at admissions, so the set can drain past r between two consecutive looks. Section 5.9 measures that at every transition.

4.6.5. Signal sampling

The oracle samples the dirty fraction inside `assignMode`, that is, once per admission of a root transaction rather than once per unit of simulated time. The simulation’s own tick does not drive the estimate at all.

Under EQUILIBRIUM the transaction population is held constant, so admissions track completions and the oracle’s sampling rate is approximately the system’s throughput. The filter’s time constant is expressed in samples, the quantity it estimates evolves in seconds, and the exchange rate between them is the throughput that the oracle itself suppresses whenever it routes work to ACID. Across the engaged cells of the X2 grid the measured interval between samples ranges from 0.8 s to 228 s, the long end being ACID at one reconciler; Table 5.16 gives the $c = 0.10$ slice. This is a design property of the implementation; its consequences are measured in Section 5.9.

4.7. Implementation notes

Hardening the received code surfaced several defects that would have affected the measurements. Each is documented with its symptom, the fix, and the metric it would otherwise have distorted, with the detail placed in the appendix.

5. Experiments

The hybrid mode is evaluated with five parameter sweeps, 640 simulation runs in total. The first three change the system itself and the last two change the workload it has to handle:

- **X1 — reconciler processors** (Section 5.3). Varies how much capacity is available for reconciliation. This shows in which range of settings the adaptive oracle switches modes at all, and what running a mixture of the two modes costs compared with running either mode alone.
- **X2 — critical threshold** (Section 5.4). Varies the threshold c at which the oracle turns ACID on. This tests whether c behaves as a consistency budget, that is, whether the measured inconsistency stays below the value that was configured, and what throughput is given up for a tighter budget.
- **X3 — adaptive versus random oracle** (Section 5.5). Replaces the measured dirty fraction by a fixed probability: each transaction is assigned a mode by a coin flip instead of by the state of the system. Both oracles can reach the same ACID share, so the comparison isolates what the feedback signal contributes from what simply running some ACID contributes.
- **X4 — semantic overlap** (Section 5.6). Varies how much of the system one write reaches, by changing the fraction of resources tied to resources in other models, and therefore how much work one write creates elsewhere.
- **X5 — resources per transaction** (Section 5.7). Varies how many resources a single transaction touches, and therefore how deep one unit of submitted work reaches into the schema.

The five sweeps answer RQ2 and RQ3. RQ1 is not among them, and it is worth saying why rather than leaving the reader to notice the omission. RQ1 asks how a hybrid mode can be integrated safely into a multi-model transactional system, which is a design question: it is settled by constructing something that works and arguing that it is correct, not by varying a parameter and measuring the result. Its answer is therefore Chapter 4 — the mixing hazard, the locked-XOR-dirty rule that resolves it, and the argument that the rule adds no deadlock — and no sweep can confirm or refute it in the way a sweep settles a question about throughput or inconsistency.

What experiments can do is show that the design behaves as claimed once it is running. Section 5.2 does this before the sweeps begin: it reports how often the mixed state was actually reached during the 640 runs and resolved by the rule, which establishes that the

guarantee is not vacuous. It is a validation of the construction rather than a sweep, which is why it carries no parameter of its own. Every later section then depends on it, because a performance number measured under a rule that was not well defined would mean nothing.

The remaining questions map onto the sweeps directly. RQ2, which compares the hybrid mode with pure ACID and pure BASE and asks whether the inconsistency it permits can be configured, is answered by X1 and X2, with X4 and X5 establishing the range of workloads over which that configurability holds. RQ3, which asks whether choosing the mode from the system’s own metrics beats a random assignment, is answered by X3.

5.1. Experimental settings

5.1.1. Simulator baseline configuration

Every experiment varies one or two parameters and holds everything else at the baseline of Table 5.1.

Table 5.1.: Baseline configuration.

Parameter	Baseline value	What it controls
<i>System</i>		
Resources N	5000	Size of the data set
Models	10	Number of co-existing data models
Semantic overlap σ	0.30	How much information the models share (X4)
IO processors	30	Capacity for IO work
CPU processors	30	Capacity for CPU work
Locks	5000	Available locks
Reconcilers R	swept	Processors reserved for reconciliation (X1)
CPR strategy	ON_CASCADE	When cross-model repairs are triggered
maxBookkeeperDepth	5	How far a repair may cascade
<i>Workload</i>		
Profile	EQUILIBRIUM	Keeps the number of in-flight transactions fixed
In-flight transactions	30	The constant that the profile holds
Resources per transaction L	50	Most resources one transaction touches (X5)
Models per transaction	up to 4	Models one transaction may span
<i>Oracle</i>		
Critical threshold c	0.10	Dirty fraction at which ACID is switched on (X2)
Recovery threshold r	0.05	Dirty fraction at which BASE is resumed (Sec. 5.9)
<i>Run</i>		
Duration	50 000 s	Simulated time per run
Seed pairs	5	Replicates per configuration

The oracle compares the two thresholds against the dirty fraction as it is at the moment of the decision; there is no filter and no further tuning parameter (Section 5.8). The two are not realised equally well: c is met closely, while r turns out to be an upper bound on where an

ACID excursion ends rather than the point at which it ends. That asymmetry applies to every experiment below, so it is reported once, at the end of the chapter (Section 5.9).

The simulator generates its own workload, so no external data set is involved. The EQUILIBRIUM profile keeps the number of in-flight transactions constant: whenever a transaction completes, a new one is admitted. The load is therefore stationary by construction, which is what makes throughput comparable across configurations, and also what prevents this chapter from testing adaptation to a load that changes over time (Section 5.10).

One value deviates from the reference configuration of the base simulator: a run lasts 50 000 s of simulated time rather than 10 000 s. The reason is that switching modes is a slow event. The oracle only looks at the dirty fraction when a new transaction is admitted, and by the time the system enters ACID mode a large share of the resources is already dirty, so the ACID transactions that follow have to wait for resources that are still being repaired. Getting back out of ACID mode is then limited by how much capacity is available for reconciliation, which at the low end of the sweep is a single processor. A run therefore stays in one mode for long stretches, and a 10 000 s run would contain only a handful of switches — too few to read a stable 95th percentile from. At 50 000 s every run in the range where the oracle is active contains enough of them for the tail to be meaningful.

5.1.2. Implementation and execution

Each experiment is driven by its own runner script. The script takes the parameters that the experiment varies, runs every combination of them under five seed pairs, and writes one row per run into a CSV file. The exported result files and the analysis scripts behind the tables in this chapter are part of the supplemental material.

Two seeds define a system instance. The *semantic-overlap seed* fixes which resources of which models represent the same information, and the *transaction-sampling seed* fixes the arrival sequence: how many resources each transaction touches, which ones, and in which models. The same five seed pairs are used for every mode in every experiment, so ACID, BASE and both HYBRID configurations see exactly the same transactions on exactly the same overlap structure; the pairs themselves are listed with the supplemental material. All tables report the mean over the five replicates. Because the simulator's clock is virtual and all seeds are fixed, a run reproduces exactly, independently of the host machine.

5.1.3. Metrics

Three numbers carry the argument, and each of them is reported in almost every table of this chapter.

Throughput (`throughput_tps`) — root transactions completed per second of simulated time.

This is the performance side of the trade-off: keeping the data consistent is only useful if the system still does work.

p_{50} **dirty fraction** (`dirty_p50_pct`) — the median share of resources that are inconsistent.

This describes the typical state of the system.

p_{95} **dirty fraction** (`dirty_p95_pct`) — the 95th percentile of the same quantity, that is, the value that inconsistency stays below for 95% of the simulated time. This describes the bad case, and it is the number a threshold is supposed to bound: a mixture of modes can hit any average, so only the tail shows whether the budget actually holds.

Both percentiles are taken over samples spaced evenly in simulated time, so they are statements about time and not about transactions. Sampling happens after the work of each tick, so a reading is always a post-repair value and short peaks inside a tick are not seen. Pure ACID runs have no dirty-fraction data at all, since inconsistency cannot occur there by construction; those cells read “n/a by construction”.

Two further numbers say how much ACID a run actually performed, and they are not interchangeable:

ϕ_{time} (`acid_fraction_time`) — how much of the run the system spent in ACID mode. The oracle timestamps every decision and attributes the interval since the previous one to the mode that was in force across it, so the total ACID time is known exactly; divided by the run duration it gives ϕ_{time} . The accounting is the same code for both oracles, so the column is comparable between them.

ϕ_{comp} (`acid_fraction_completed`) — how much of the finished work ran under ACID: the number of root transactions completed in ACID mode, divided by all completed root transactions.

The two can differ widely, because one ACID transaction occupies far more time than one BASE transaction.

[(TODO: decide where this belongs)] Bookkeeper transactions are not counted in ϕ_{comp} . They inherit the mode of the root transaction that created them, so counting them would weight each mode by how many repairs it happened to trigger rather than by the work that was actually submitted.

[(TODO: two different samplers -> own chapter)] Both of these numbers, and the dirty fractions above, come from reading the state of the system while it runs — but not every reading is taken by the same reader, and that distinction is what Section 5.9 turns on. The *signal* sampler is the oracle’s sensor: it is reached only when a root transaction arrives without a mode tag, so its estimate advances once per admission, on no timer, at whatever rate the workload happens to produce. The *metrics* sampler is the measuring instrument: once per simulated second, regardless of mode. The sensor is therefore load-gated — its sampling rate is roughly the throughput that its own decisions suppress. Clocking the instrument the same way would make the two indistinguishable, since an apparent shift in the operating point could equally be the instrument slowing down together with the system it measures.

Two further columns are used wherever the load on reconciliation is at issue. `dirty_mark_events_total` counts transitions of a resource from clean to dirty, so over the run duration it is the rate at which repair work is created; `dirty_clear_events_total` counts repairs completed, so over the duration it is the rate at which that work is served, against a ceiling of 0.5R resources per second (Section 5.9). A write to a resource that is already dirty is counted in

`dirty_mark_calls_total` but creates no event and no additional repair, which is why the two mark columns differ.

Transaction latency (`txn_latency_mean_ms`) is the time from a transaction's creation to its commit, recorded for completed root transactions only; bookkeepers are excluded, as they are everywhere else. It is the measure of how long work sits in the system, and X5 (Section 5.7) uses it because there a transaction is not a fixed amount of work: the simulator charges processing time per resource touched, 200 ms of IO and 100 ms of CPU each, so a larger transaction occupies the system proportionally longer.

5.2. X0: Arbiter validation

RQ1 asks how a hybrid mode can be integrated *safely*. The answer is given by construction in Section 4.4: the locked-XOR-dirty invariant holds because neither write path can produce a violating state. An ACID transaction cannot acquire a lock on a dirty resource, and a BASE transaction cannot mark a locked resource dirty; each defers instead. What construction alone does not show is that the situation ever arises. A rule that is never reached is trivially never violated, and the guarantee would then be empty. X0 reports the instrumentation that distinguishes the two.

Both deferral paths are counted per run. The counter `acid_txn_blocked_on_dirty_count` records how often an ACID transaction was made to wait for reconciliation to clear a resource it needed, and `base_txn_yielded_on_locked_count` how often a BASE transaction abandoned an update because the resource was locked. Every increment is one occurrence of the mixing hazard of Section 4.3, resolved by the rule. X0 is not a sweep of its own: it aggregates these two columns over all 640 runs of the five sweeps reported in this chapter.

Table 5.2.: X0: arbiter activity across all runs in this chapter, by execution mode.

Mode	Runs	ACID waits on dirty	BASE yields on locked
ACID	35	0	0
BASE	35	0	0
HYBRID_ADAPTIVE	435	550 793	4 684 097
HYBRID_RANDOM	135	121 021	1 078 556
Total	640	671 814	5 762 653

Source. Columns `acid_txn_blocked_on_dirty_count` and `base_txn_yielded_on_locked_count`, summed over all CSVs of the five sweeps (X1–X5), five seeds each.

A1 The counters are zero in every pure run. All 35 pure ACID and all 35 pure BASE runs record zero on both counters, without exception. The arbiter is inert exactly where only one set of semantics is in use, which is what confirms that the counters measure cross-mode interaction and not ordinary lock or CPR activity.

A2 The hazard is the normal case, not a corner case. Across the hybrid runs the rule fired 6.4 million times, 5.8 million of them on the BASE side. The state that neither pure

mode defines is therefore reached constantly under mixed execution, and the invariant holding is a claim with content rather than a claim about a situation that never occurred.

A3 Arbiter activity coincides exactly with oracle engagement. Of the 570 hybrid runs, 102 record zero on both counters, and in every one of those runs `mode_switch_count` is also zero; the correspondence is exact in both directions, with no exceptions among the 570. These runs are the configurations independently identified as degenerate elsewhere in this chapter: $R = 0$ and $R = 25$ in X1 (Section 5.3.2), c above the BASE steady-state dirtiness in X2 (Section 5.4.2), and overlap below the engagement threshold in X4 (Section 5.6.1). Degeneracy is thus visible in a quantity derived from the arbiter alone, independently of the switch count used to establish it elsewhere.

What this does not establish. X0 is evidence for the invariant, not for the deadlock-freedom argument of Section 4.4.2, which remains analytical. Every run in this chapter ends by reaching the fixed simulated horizon of 50 000 seconds rather than by quiescing, so termination is imposed by the harness and a stalled run would end the same way; the run data cannot separate the two. The weaker statement it does support is that no run stopped making progress: the least productive hybrid run in the whole set still committed 336 transactions, and every configuration retains nonzero throughput in Tables 5.3 and 5.5. A direct test would need a wait-for-graph check at runtime, which is left to future work.

5.3. X1: Reconciler-processor sweep

X1 answers the first half of RQ2: how the hybrid mode compares with pure ACID and pure BASE in throughput and inconsistency. It is the only sweep in which all three run under identical conditions. Before that comparison means anything, two things must be settled: whether the oracle engages at all, and what the mixture costs when it does. The other half of RQ2, configurability, is left to X2; X1 holds $c = 0.10$ throughout.

The sweep covers $R \in \{0, 1, 5, 10, 15, 20, 25\}$ across four configurations — pure ACID, pure BASE, HYBRID_ADAPTIVE and HYBRID_RANDOM — and five seeds, 140 runs.

One point about R has to be kept in mind when reading the table. Reconcilers are not extra hardware: the simulator has a fixed pool of processors, and every reconciler is taken out of it. At $R = 1$, one processor repairs and 29 are left for foreground transactions; at $R = 15$ the pool is split in half, 15 repairing and 15 running transactions. Raising R therefore buys faster repair by giving up exactly the capacity that produces the work needing repair, which is why throughput falls off towards the right-hand end of the sweep. $R = 0$ is not a smaller version of the same setting but a different arrangement altogether, and is discussed on its own below.

Table 5.3 reports one row per configuration and reconciler count. Two of its columns were introduced in Section 5.1.3: ϕ_{time} is the share of the run spent in ACID mode and ϕ_{comp} the share of completed transactions that ran under ACID. *Switches* counts how often the oracle changed the mode during the run, so a row with zero switches is degenerate — the oracle never engaged and the run is a pure BASE run under a different label.

Table 5.3.: X1: results by reconciler count R and configuration.

R	Mode	Thr. (tps)	ϕ_{time}	ϕ_{comp}	Switches	p_{50} (%)	p_{95} (%)
–	ACID (all R)	0.0418	1.0000	1.0000	0.0	n/a by construction	
0	BASE	1.9536	0.0000	0.0000	0.0	3.17	3.87
1	BASE	3.8685	0.0000	0.0000	0.0	29.50	29.64
5	BASE	3.3344	0.0000	0.0000	0.0	27.20	27.53
10	BASE	2.6681	0.0000	0.0000	0.0	23.38	23.93
15	BASE	2.0013	0.0000	0.0000	0.0	17.74	18.47
20	BASE	1.3343	0.0000	0.0000	0.0	8.10	8.93
25	BASE	0.6665	0.0000	0.0000	0.0	0.16	0.50
0	HYBRID_ADAPTIVE	1.9536	0.0000	0.0000	0.0	3.17	3.87
1	HYBRID_ADAPTIVE	0.0726	0.7776	0.0918	79.8	6.80	10.12
5	HYBRID_ADAPTIVE	0.3453	0.5068	0.0966	234.0	5.67	10.22
10	HYBRID_ADAPTIVE	0.5883	0.2886	0.0694	224.0	3.22	9.98
15	HYBRID_ADAPTIVE	0.5941	0.1667	0.0716	162.4	0.43	9.36
20	HYBRID_ADAPTIVE	1.3089	0.0025	0.0009	3.6	8.08	8.92
25	HYBRID_ADAPTIVE	0.6665	0.0000	0.0000	0.0	0.16	0.50
0	HYBRID_RANDOM	0.0849	0.7887	0.4968	2137.2	0.00	0.06
1	HYBRID_RANDOM	0.0831	0.7890	0.4974	2090.8	1.02	2.98
5	HYBRID_RANDOM	0.0849	0.7873	0.4971	2135.8	0.00	0.18
10	HYBRID_RANDOM	0.0850	0.7761	0.4969	2137.6	0.00	0.09
15	HYBRID_RANDOM	0.0876	0.6601	0.4972	2202.0	0.00	0.07
20	HYBRID_RANDOM	0.0938	0.5751	0.4978	2350.8	0.00	0.07
25	HYBRID_RANDOM	0.0987	0.5197	0.5003	2471.2	0.00	0.07

Why $R = 0$ behaves differently. The $R = 0$ row looks contradictory at first: no reconcilers, and yet the lowest dirty fraction of the whole BASE block together with a throughput in the middle of the range. Neither follows from repairs being skipped, because at $R = 0$ they are not skipped. The simulator still creates the same repair work; it simply has nowhere separate to put it, so the repairs are queued on the ordinary IO and CPU processors and run there, after the transaction that caused them has already committed (Section 2.3.1). Reconciliation at $R = 0$ is not absent, it is unpartitioned.

The difference the partition makes is not the order of work inside a repair but how many repairs can be under way at once. Every repair runs its IO phase and then its CPU phase, in that order, under both arrangements; the two never overlap within one repair.

- **Dedicated reconcilers ($R \geq 1$).** One worker owns one repair until it finishes, advancing it by a single phase per simulated step. At most R repairs are in flight, whatever the rest of the pool is doing, and each worker finishes one repair every two seconds.
- **No partition ($R = 0$).** Repairs join the ordinary IO and CPU queues, so many different repairs are in flight at once across the 30 processors — one repair’s IO on one processor while another repair’s CPU runs on another. The number in flight is bounded by the

pool rather than by R .

Two consequences follow. Repair at $R = 0$ is faster, not slower: its rate is limited only by how much of the pool the foreground leaves idle, which in this workload is more than a handful of dedicated reconcilers would provide, so the dirty fraction is correspondingly low.

Throughput is nevertheless not the highest, because for $R \geq 1$ it is set by the processors left over after the carve-out rather than by repair capacity at all — across the BASE rows it is proportional to $30 - R$ to within 0.1 %, so reconcilers cost throughput by occupying processors, not by competing for service time. At $R = 0$ no processors are lost, but foreground work and repair work share one pool, so each transaction takes roughly twice as long and throughput lands in the middle of the range. The $R = 0$ point is therefore not the left endpoint of the $R \geq 1$ curve; it is a different allocation regime plotted on the same axis, and the two should not be read as one trend.

5.3.1. Validation against the reference results

The BASE line reproduces the result of the base simulator [7]. Throughput and inconsistency move from (1.95 tps, 3.17%) at $R = 0$ to (3.87, 29.50%) at $R = 1$, then decline monotonically to (0.67, 0.16%) at $R = 25$. This is the same shape reported there: inline CPRs give the lowest inconsistency at capped throughput, a single dedicated reconciler raises throughput while inconsistency rises sharply, and many reconcilers drive inconsistency back down at a throughput cost, with foreground execution starved at the top of the range.

5.3.2. Where the oracle engages

The adaptive oracle has three regimes rather than one monotone trade-off. At $R = 0$ it is inert and degenerates exactly to BASE. Between $R = 1$ and $R = 15$ it engages substantially, with ϕ_{time} from 0.17 to 0.78 and 80 to 234 switches; this band contains the throughput valley. From $R = 20$ upwards it is inert again and throughput free-rides back to the BASE level. At $R = 0$ and $R = 25$ every HYBRID_ADAPTIVE statistic equals the corresponding BASE row in all five seeds, so those points are excluded from adaptive-versus-BASE comparisons or marked as degenerate.

O1 Reconciler count has no effect under pure ACID. Throughput (0.0418 tps) and the started and aborted transaction counts are identical across all seven values of R , confirming that the reconciliation stage is never invoked under ACID. The ACID row is a flat reference line rather than an unswept baseline.

O2 The adaptive mode degenerates to BASE at both ends, and the throughput curve between them is a valley (Figure 5.1). At $R = 0$ dirtiness never accumulates enough to cross c ; at $R = 25$ repair is fast enough that it never reaches c either (Figure 5.2). In between, throughput collapses from 1.95 tps at $R = 0$ to 0.073 tps at $R = 1$, recovers through $R = 5, 10, 15$, and is back at the BASE level by $R = 25$. The two ends look alike in the table and have different causes: at low R the oracle correctly detects a problem

that drains slowly, and pays for it, while at high R it simply never fires and the BASE throughput comes for free. Adaptivity is confined to an interior band of R .

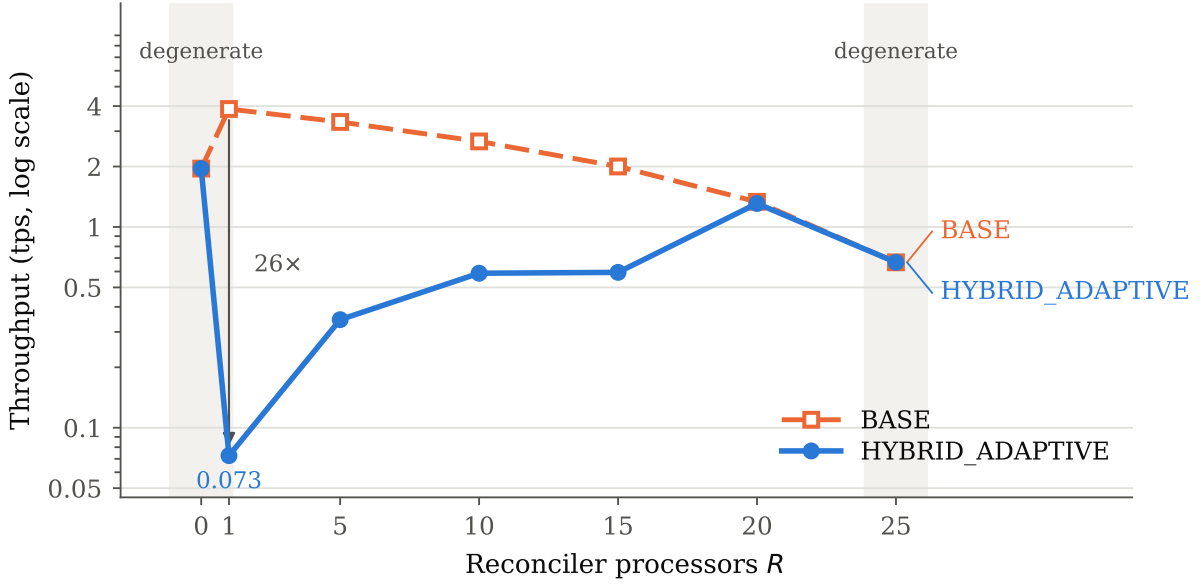


Figure 5.1.: O2: the adaptive throughput valley.

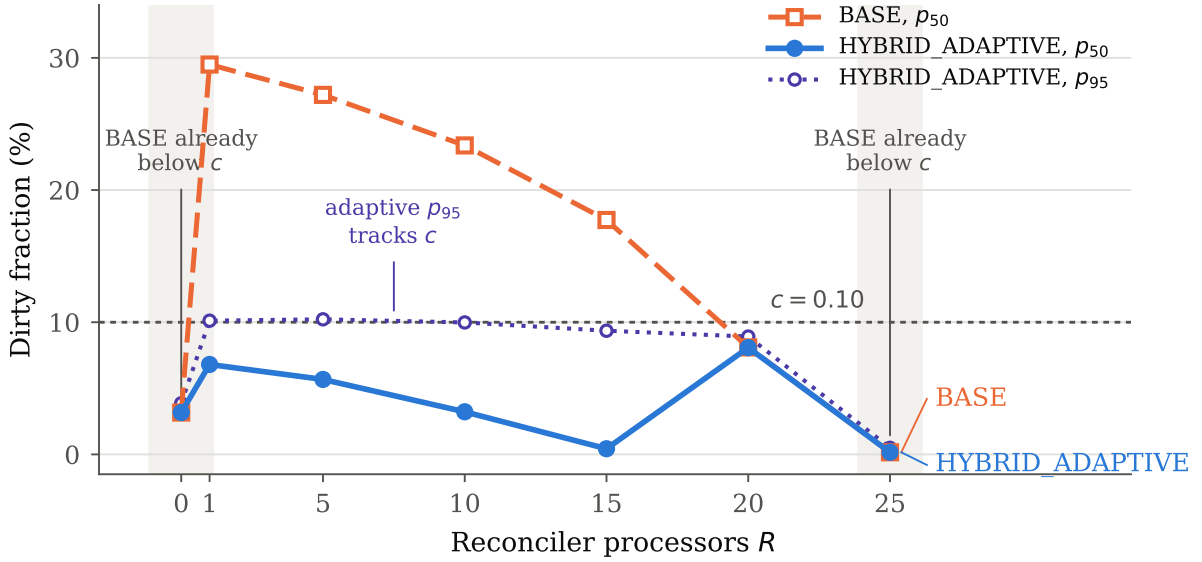
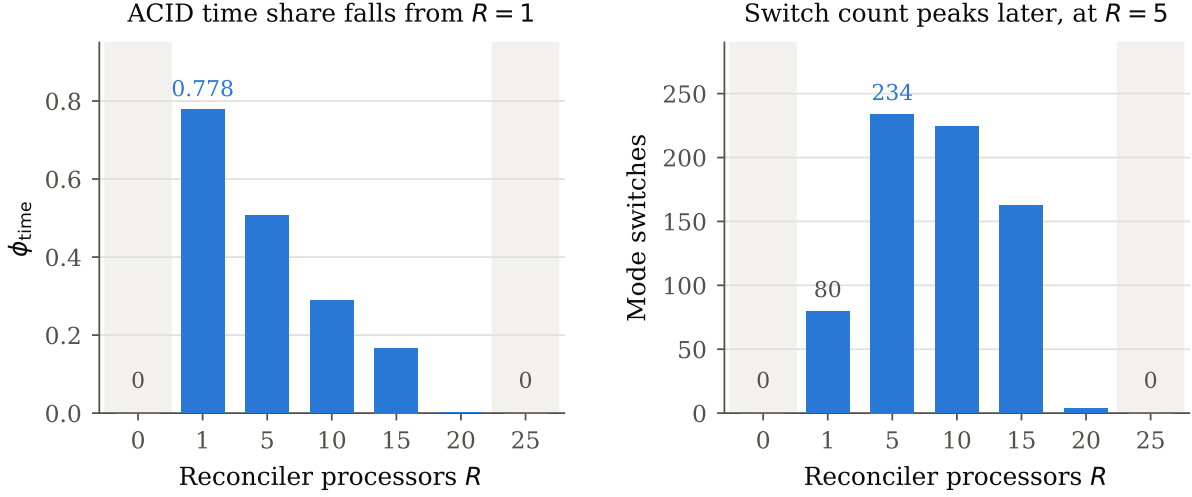


Figure 5.2.: O2: the uncontrolled dirty fraction against c , which decides whether the oracle fires at all. Dotted lines are p_{95} .

O3 ACID share and switch frequency peak at different R (Figure 5.3). ϕ_{time} falls monotonically from $R = 1$ ($\phi_{\text{time}} = 0.7776$) onwards, while the switch count peaks later, at $R = 5$

(234 against 80 at $R = 1$). $R = 1$ produces few long excursions and $R = 5$ many short ones. More switching does not imply more time in ACID.



Shaded columns are the degenerate ends $R = 0$ and $R = 25$, where the oracle never fires.

Figure 5.3.: O3: the ACID time share and the switch count peak at different R .

O4 The random oracle is an effective negative control (Figure 5.4). With routing independent of the dirty fraction, throughput is nearly flat, 0.083 to 0.099 tps across the entire R range, so the valley in O2 belongs to the feedback loop and not to reconciler provisioning as such. The quantity that stays pinned is ϕ_{comp} , at 0.4968 to 0.5003, which is the coin flip behaving as specified, while ϕ_{time} falls from 0.7887 at $R = 0$ to 0.5197 at $R = 25$: the random oracle controls its routing share exactly and its time share not at all.

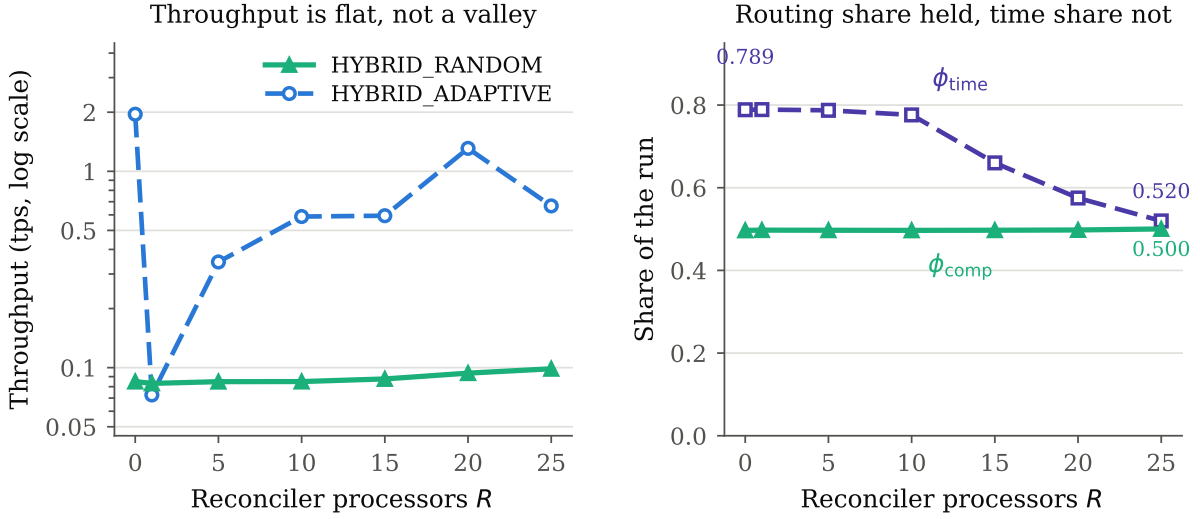


Figure 5.4.: O4: the random oracle as a negative control, flat where the adaptive oracle has a valley.

5.3.3. Is the hybrid a weighted average of its two modes?

If the hybrid mode were nothing more than the two pure modes taken in turn, its throughput would follow from how long it spends in each. A run that spends a share ϕ_{time} of its time in ACID and the rest in BASE would deliver

$$T_{\text{pred}}(R) = \phi_{\text{time}} \cdot T_{\text{ACID}} + (1 - \phi_{\text{time}}) \cdot T_{\text{BASE}}(R), \quad (5.1)$$

with $T_{\text{ACID}} = 0.0418$ tps and $T_{\text{BASE}}(R)$ taken from the BASE rows of Table 5.3. This is the natural first guess, and it is worth writing down because it is exactly what switching between two modes would give if the switching itself cost nothing.

It costs something. Table 5.4 puts Equation 5.1 next to what the runs actually delivered, and the two disagree wherever the oracle is active. At $R = 1$ the weighted average predicts 0.893 tps and the run delivers 0.073, short by a factor of 12.3. The gap narrows as R rises — $4.8\times$ at $R = 5$, $3.2\times$ at $R = 10$, $2.8\times$ at $R = 15$ — but it never closes. The hybrid stays ahead of pure ACID throughout, so mixing is not simply the worst of both; it lands far below the blend of the two that its own time split predicts.

Two checks say the shortfall is real rather than an artefact of the prediction. The first is that the prediction is accurate where it should be: at $R = 20$ the oracle barely engages ($\phi_{\text{time}} = 0.0025$) and Equation 5.1 lands within 2% of the measurement. A formula that missed everywhere would be the wrong yardstick; this one is right when there is almost nothing to mix, which is what makes its failure elsewhere worth reporting.

The second is that the shortfall does not depend on which oracle does the mixing. The random oracle shows it too: at $R = 10$ it spends $\phi_{\text{time}} = 0.7761$ of the run in ACID, which predicts 0.630 tps against a measured 0.0850 tps, a gap of $7.4\times$. That oracle ignores the dirty

Table 5.4.: HYBRID_ADAPTIVE throughput as measured, against the weighted average of Equation 5.1.

R	ϕ_{time}	BASE (tps)	Predicted	Observed	Gap
1	0.7776	3.868	0.893	0.073	$12.3\times$
5	0.5068	3.334	1.666	0.345	$4.8\times$
10	0.2886	2.668	1.910	0.588	$3.2\times$
15	0.1667	2.001	1.675	0.594	$2.8\times$
20	0.0025	1.334	1.331	1.309	$1.0\times$

Source. Derived from Table 5.3; pure-ACID throughput 0.0418 tps from the same sweep.

fraction entirely, so the cost belongs to running the two modes together and not to the rule that chooses between them.

What Equation 5.1 leaves out is the cost of the switch itself. An ACID transaction admitted while the resources it needs are still dirty has to wait for them to be repaired (Section 4.4.3), and that waiting is invisible to a calculation built only from how long each mode ran. Section 5.4.3 measures it.

5.4. X2: Consistency-budget sweep

The setting this experiment is about the critical dirty-fraction threshold c in the simulator (Table 2.1). It is the share of resources allowed to be inconsistent before the oracle switches the system to ACID: at every admission the oracle compares the live dirty fraction d_k against c , and enters ACID mode as soon as $d_k \geq c$ (Equation 4.3). It is expressed as a fraction of all resources, so $c = 0.10$ means “switch to ACID once a tenth of the resources are waiting to be repaired”. It is the setting through which an operator states how much inconsistency the system may carry.

X1 establishes that the oracle engages and what happens when it does, but it holds this threshold fixed at $c = 0.10$, so it says nothing about whether the value of c makes any difference. That is the claim the thesis rests on: that an operator can set a level of inconsistency and get it. X2 tests it by varying c directly and asking whether the measured inconsistency follows.

The sweep covers $c \in \{0.02, 0.05, 0.10, 0.15, 0.20, 0.30\}$ against $R \in \{1, 5, 10, 15\}$, five seeds, 120 runs, all HYBRID_ADAPTIVE. The recovery threshold is held at $r = 0.5c$ throughout, so the hysteresis band scales with the budget and the sweep stays one-factor. The values $R \in \{0, 20, 25\}$ are excluded because X1 shows the oracle is inert there.

One property of the baseline sets the top of the useful range for c . The semantic overlap is held at $\sigma = 0.30$, so 30% of the 5000 resources carry a cross-model rule and the rest have nothing to reconcile and can never be marked dirty. The dirty fraction therefore cannot get much above 30% however the system is run, and the measurements agree: the highest value recorded anywhere in the 120 runs is 29.88%. That is why $c = 0.30$ sits at the top of the sweep. It is not a value an operator would pick but the point at which the budget reaches the limit

of the system, and it is degenerate at every reconciler count for that reason. X4 (Section 5.6) varies σ itself and shows what happens to the budget when that limit moves.

The reason for varying the reconciler count R as well as c is that a setting is only useful if it works the same way on differently provisioned systems. If the same c gave a different result on a system with one reconciler than on a system with fifteen, an operator would have to re-tune it for every deployment, and it would not be a setting in any practical sense.

5.4.1. The critical threshold as a control knob

Table 5.5.: X2: measured p_{95} dirty fraction against the configured threshold.

Configured c (%)	$R = 1$	$R = 5$	$R = 10$	$R = 15$	Range (pp)
2	2.52	2.32	2.10	1.84	0.68
5	5.43	5.36	5.14	4.71	0.72
10	10.12	10.22	9.98	9.36	0.86
15	14.76	14.79	14.56	14.28	0.51
20	19.52	19.54	19.44	deg.	0.10
30	deg.	deg.	deg.	deg.	–

Table 5.5 reports one measured p_{95} dirty fraction per cell, in percent and averaged over the five seeds. Each number should be read against the configured value in the first column of the same row. “deg.” marks a degenerate cell: there the threshold sits above the inconsistency the system produces on its own, the oracle never switches to ACID, and the measured value is simply the BASE level at that reconciler count. The last column is the range across the four reconciler counts in that row — the largest measured value minus the smallest — counting only the cells where the oracle engaged.

This is the central result of the thesis. The configured threshold is varied from 2% to 30% and the reconciler count from 1 to 15, and in every cell where the oracle engages the measured p_{95} stays within 0.74 percentage points of the configured c — not only on average, but in each of the 95 individual engaged runs. The largest single deviation anywhere is -0.74 pp. At $c = 0.10$, the twenty individual runs land between 9.32% and 10.28%. Figure 5.5 plots the same data as a distance: how far each cell landed from the value it was set to, in percentage points. In the terms of Equation 1.1, c behaves as a bound on the tail that can be configured: it is set in the same units as the quantity it controls, and the measured value follows it without having to be re-tuned per configuration.

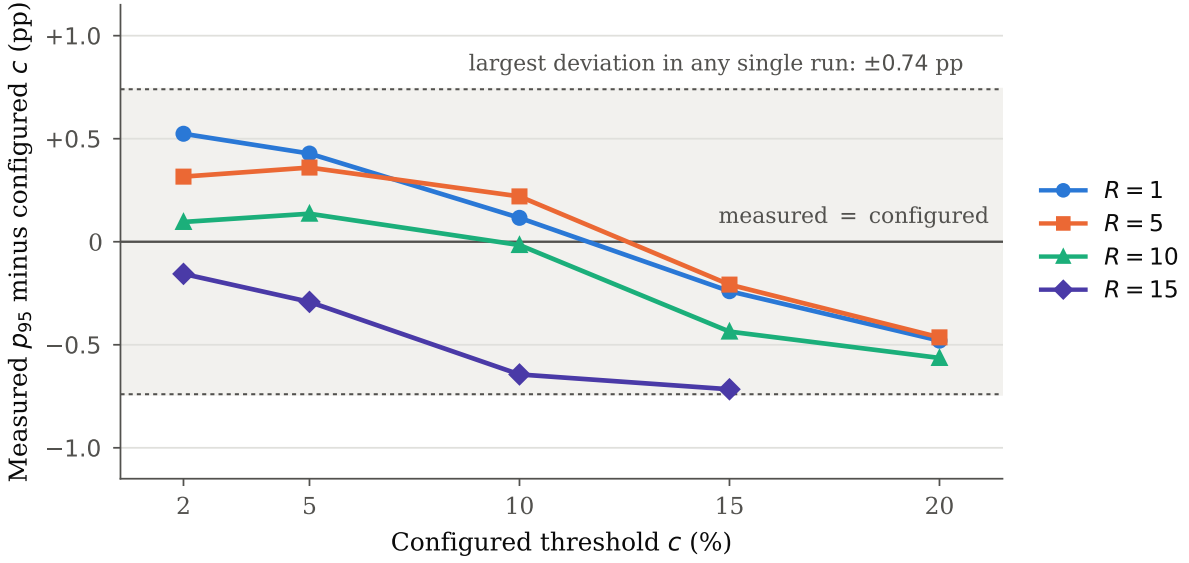


Figure 5.5.: How far the measured p_{95} dirty fraction lands from the configured threshold c , one line per reconciler count. Zero is a run that hit its setting exactly. The band marks the largest deviation seen in any single run, ± 0.74 pp. Degenerate cells are left out.

The threshold is a bound, not a hard ceiling. The measured value lands close to c but not exactly on it, and it can land on either side. Reading the grid, the deviation is $+0.52$ pp at the top-left corner ($c = 0.02$, $R = 1$) and -0.72 pp at the bottom-right ($c = 0.15$, $R = 15$), crossing zero in between. The two ends have different causes.

Going over c is a matter of when the oracle looks. There are two ways work enters the system, and only one of them is a moment at which the mode is decided. A root transaction is tagged at admission, and that is when the oracle reads the dirty fraction. Everything else — the bookkeeper transactions that a root transaction spawns — inherits the tag of its root and never consults the oracle (Section 4.6.1). So the dirty fraction is only checked at root admissions, while work that dirties resources keeps arriving in between. When root admissions are rare, which is what a low reconciler count produces, the dirty fraction can climb past c before the oracle gets its next look. **[*TODO: two admissions — root admissions sample the oracle, inherited bookkeeper work does not; add the reference here once the main discussion of this is written*]**

Staying under c has the opposite cause. Once the system is in ACID mode repair is quick, so by the time the oracle next looks, the dirty set has usually already fallen well below the recovery threshold r , and the run spends most of its time comfortably under the budget rather than at it. Section 5.9 measures this at every transition.

The right way to state the result is therefore as a magnitude, $|p_{95} - c| \leq 0.74$ pp, and not as an offset in one direction. The bound is what an operator can rely on; which side of c the run lands on depends on the configuration, and both sides are small.

5.4.2. Findings

- O5 The deviation moves from overshoot to undershoot as the budget grows** (Figure 5.5). At every reconciler count the difference between the measured p_{95} and the configured c falls steadily as c rises. At $R = 1, 5$ and 10 it starts positive at the tight budgets and ends negative at the loose ones, crossing zero near $c = 0.10$; at $R = 15$ repair is fast enough that it is already negative at the tightest budget. The more reconcilers there are, the earlier the crossing happens. The two ends have the causes given in Section 5.4.1: with a tight budget and slow repair the dirty fraction can climb past c between two root admissions, while with a loose budget and fast repair the dirty set is already well below the threshold by the time the oracle looks again. Neither side is large — every run stays inside the 0.74 pp bound — but the sign is predictable rather than random, so an operator setting a small c should expect to land a little above it, and one setting a large c a little below.
- O6 Setting the threshold too high switches the oracle off.** Five of the 24 cells are degenerate: the system is repaired faster than it can get dirty enough to reach c , so the oracle never fires and the run is pure BASE. This happens exactly when c is above the inconsistency BASE produces on its own at that reconciler count, that is when $c \geq p_{95}^{\text{BASE}}(R)$, which accounts for all five. At $R = 10$ BASE alone runs at 23.93%, so $c = 0.30$ never fires; at $R = 15$ BASE alone runs at 18.47%, so it stops firing one step earlier, at $c = 0.20$. The same rule is used again in X4 (Section 5.6). Degenerate cells have to be spotted before any average is taken over a sweep, which is why every HYBRID row in this thesis reports its switch count.
- O7 Buying more throughput with the budget works, but it is a poor exchange** (Table 5.6, Figure 5.6). Allowing more inconsistency does let more work through, because the system spends less of its time in the slow mode. What it does not do is pay back in proportion: ten times the budget returns only one to two and a half times the throughput, and the whole range stays far under what the same system reaches without the oracle at all — between a fiftieth and a third of the pure BASE throughput at the same reconciler count. At $R = 1$ the exchange all but fails: throughput dips before it recovers and ends a fifth above where it started, 0.1083 tps against 0.0907 tps, for ten times the inconsistency. So the threshold is a real control, but an operator cannot expect to trade inconsistency for speed at anything like a fair rate. Section 5.4.3 explains where the throughput goes.

Table 5.6.: X2: mean throughput and mode-switch count against the configured threshold.

c (%)	Throughput (tps)				Mode switches			
	$R=1$	$R=5$	$R=10$	$R=15$	$R=1$	$R=5$	$R=10$	$R=15$
2	0.0907	0.2551	0.3951	0.3793	302	509	443	238
5	0.0718	0.3029	0.4860	0.4769	152	347	304	198
10	0.0726	0.3453	0.5883	0.5941	80	234	224	162
15	0.0840	0.3973	0.6895	0.7781	51	162	179	131
20	0.1083	0.5080	0.9460	deg.	36	108	124	0
30	deg.	deg.	deg.	deg.	0	0	0	0

Source. Same source as Table 5.5; columns `throughput_tps` and `mode_switch_count`. “deg.” is a degenerate cell, and a switch count of zero marks the same cells.

O8 A larger budget makes the oracle switch mode less often (Table 5.6). Switch counts fall as c rises at every reconciler count. A higher threshold is crossed less often, so the run is made of fewer and longer ACID periods rather than many short ones.

Both halves of Table 5.6 move in one direction and neither moves far. Reading a throughput column downwards, every reconciler count gains from a larger budget except $R = 1$, which stays flat, and the gain is small next to the change in the setting that produced it, which is 15 times from end to end (Figure 5.6). Reading a switch column downwards, the oracle intervenes steadily less often as the budget grows.

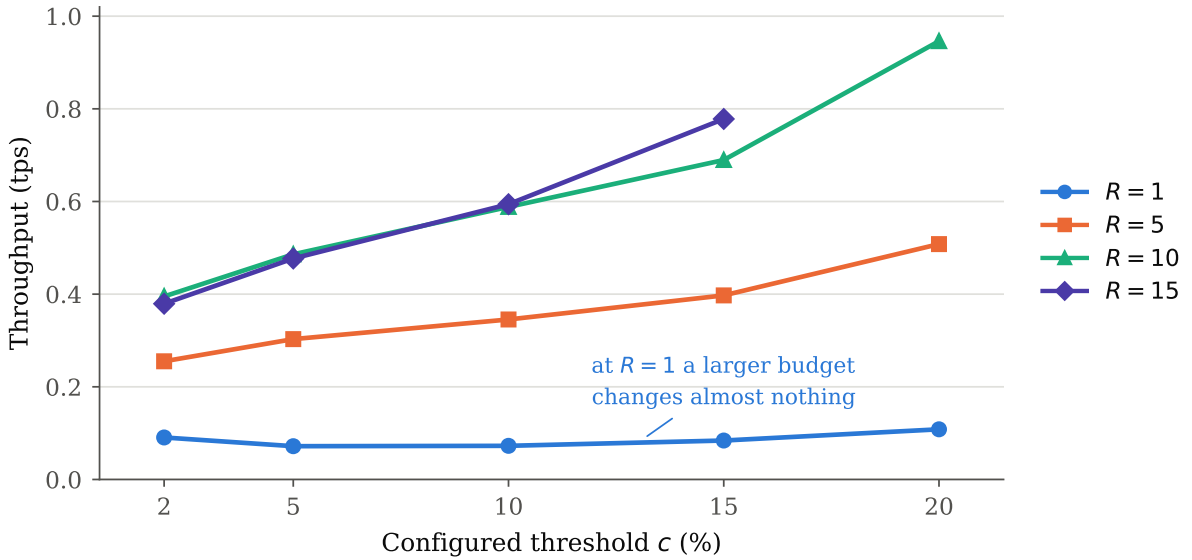


Figure 5.6.: O7: throughput against the configured threshold c , one line per reconciler count. Degenerate cells are left out, which is why the $R = 15$ line stops at $c = 0.15$.

5.4.3. The budget shifts work out of ACID, it does not make ACID cheaper

O7 is surprising once Equation 4.2 is taken into account. A transaction touches $L = 50$ resources, so its chance of meeting at least one dirty resource climbs steeply with the dirty fraction: about 64% at 2% dirty, 92% at 5% and 99.5% at 10%. A tight budget should therefore be the cheap setting — it keeps the system in the region where an ACID transaction has a real chance of finding everything it needs already clean, while a loose budget puts it in the region where it almost never does. The measurements say otherwise: at every reconciler count above one, the tight budgets are the slow ones.

The reason is that raising c does not make ACID work cheaper. It makes less of the work ACID. Each individual ACID transaction gets slower as the budget grows — there is more dirt to wait on, exactly as the model predicts — but it is a shrinking share of what the system finishes, and the run is carried instead by BASE transactions that never wait on a repair at all. Table 5.7 is arranged to show both movements at once: the left half is the ACID transactions — how many there are and what each one costs — and the right half is what the run as a whole does.

Table 5.7.: X2: what an ACID transaction costs, and how much of the run is ACID.

R	c	ACID transactions		The run as a whole			
		Adm. A	Mean wait (s)	ϕ_{time}	ϕ_{comp}	Mean lat. (s)	Thr. (tps)
1	0.02	2691	106.5	0.837	0.472	230.7	0.0907
1	0.05	1105	225.9	0.822	0.243	299.1	0.0718
1	0.10	384	392.5	0.778	0.092	261.8	0.0726
1	0.15	199	538.0	0.734	0.043	208.5	0.0840
1	0.20	123	686.5	0.665	0.021	146.4	0.1083
5	0.02	2788	18.1	0.440	0.167	78.7	0.2551
5	0.05	2735	40.2	0.487	0.147	69.9	0.3029
5	0.10	1924	70.2	0.507	0.097	67.2	0.3453
5	0.15	1186	95.0	0.481	0.054	60.4	0.3973
5	0.20	719	120.8	0.403	0.026	47.8	0.5080
10	0.02	2805	7.9	0.271	0.108	50.6	0.3951
10	0.05	2741	18.9	0.277	0.088	42.4	0.4860
10	0.10	2513	32.5	0.289	0.069	37.4	0.5883
10	0.15	1979	43.4	0.286	0.048	33.4	0.6895
10	0.20	1320	59.0	0.243	0.024	26.3	0.9460
15	0.02	2726	5.5	0.161	0.117	56.2	0.3793
15	0.05	2703	13.2	0.156	0.095	45.7	0.4769
15	0.10	2504	22.5	0.167	0.072	38.2	0.5941
15	0.15	2045	30.5	0.155	0.044	30.6	0.7781

Source. Same source as Table 5.5; columns `acid_txn_admitted_count`, `acid_dirty_block_time_mean_ms`, `acid_fraction_time`, `acid_fraction_completed` and `txn_latency_mean_ms`. Every engaged cell of the sweep is listed; the five degenerate cells have no ACID admissions and are omitted.

Reading the table. Rows are grouped by reconciler count, and within a group the budget increases downwards. A is how many root transactions were admitted in ACID mode. *Mean wait* is the average length of one wait on a repair. A transaction is parked in one of two places: at admission, before it takes any locks, or mid-flight, when a bookkeeper transaction it spawned finds a resource that has turned dirty since (Section 2.2.1). Both are timed from parking until the reconcilers release the resource, and the average is taken per wait rather than per transaction, since one transaction can be made to wait more than once. ϕ_{time} and ϕ_{comp} are the two ACID shares of Section 5.1.3 — of the run’s time and of its finished transactions — and they are not interchangeable: the first says how long the system stayed in the slow mode, the second how much of the finished work was done there. *Mean lat.* is the average end-to-end time from admission to commit, over all completed transactions of either mode.

What the table shows. The left half is the whole of the first claim. At $R = 10$ the mean wait stretches from 7.9 s to 59.0 s as the budget is raised, so ACID work gets *worse*, not cheaper — and there is steadily less of it, with admissions falling from 2805 to 1320 over the same rows. Every reconciler count behaves the same way. The right half says where the extra throughput comes from anyway: ϕ_{comp} falls monotonically at every reconciler count — 0.472 to 0.021 at $R = 1$, 0.108 to 0.024 at $R = 10$ — so the finished work is increasingly BASE work. Because an ACID transaction occupies far more time than a BASE one, that shift pulls the mean latency down (50.6 s to 26.3 s at $R = 10$), and under EQUILIBRIUM the in-flight population is held constant (Section 5.1.1), so a transaction that finishes sooner frees its slot sooner and throughput rises.

ϕ_{time} is what makes the exchange a poor one. Where ϕ_{comp} collapses, ϕ_{time} barely moves: at $R = 10$ it stays between 0.24 and 0.29 across the whole budget range, and at $R = 15$ between 0.155 and 0.167. The system is in ACID about as often as before; it simply gets less done while it is there. The operator pays real inconsistency and receives a reallocation of work from ACID to BASE inside an ACID occupancy that stays where it was — which is why the gain is real but small, and why it cannot keep going once ϕ_{comp} is near zero.

$R = 1$ is the case where even that fails. There ϕ_{time} does fall usefully, from 0.837 to 0.665, but repair is too slow for BASE to take up the slack: ACID completions collapse from 2138 to 116 while BASE completions rise only from 2395 to 5298. That is why a larger budget buys so little throughput in general, and nothing at all at $R = 1$.

5.5. X3: Removing the feedback signal

X3 is a single-variable removal. The random oracle keeps everything the adaptive one has — the arbiter, per-transaction tagging, the two dispatchers — but decides ACID or BASE by a coin flip with fixed probability p instead of by reading the dirty fraction. If the X2 results survive that removal, they did not come from the feedback loop. Removing that one input has consequences beyond the decision itself: without a signal to hold it in ACID until the dirty set has drained, the random oracle also stops collecting its ACID time in long excursions. That is a downstream effect of the removal, not a second thing removed.

One property of the workload should be stated before the results rather than after them. Every run in this thesis is executed under EQUILIBRIUM, a stationary load (Section 5.1.1). Feedback control earns its keep when the operating point moves, and a constant one cannot reward it for tracking. The throughput comparison below is therefore run in the regime least favourable to the adaptive oracle, and a null result there is the expected one rather than a surprising one. What that buys is a clean attribution: whatever advantage adaptive control turns out to have under moving load (Section 6.3) cannot be credited to hybrid execution in general, because hybrid execution on its own is measured here and does not supply it.

The sweep runs $p \in \{0.05, 0.10, 0.15, 0.25, 0.50\}$ against $R \in \{1, 5, 10, 15\}$, five seeds per cell, 100 runs, everything else at the baseline of Table 5.1. The grid is concentrated below 0.25 rather than centred on 0.5 because X1 shows the adaptive oracle finishing between 7% and 10% of its work in ACID (Table 5.3), and a grid centred on 0.5 would never bracket that. Pure BASE and pure ACID are the $p = 0$ and $p = 1$ anchors, and are already measured in X1. The adaptive comparison points are the 19 engaged cells of X2.

5.5.1. Results of the probability sweep

The random oracle controls what it reads. Compare the ϕ_{comp} entries for one p across the four blocks and they barely move: at $p = 0.05$ the realised ACID share runs 0.0445 to 0.0487, a spread of 9%, and at $p = 0.50$ it runs 0.4969 to 0.4974, a spread of 0.1%. Nothing about the system enters that number. X1 reports the same thing at a single probability (O4); the sweep shows it holds at all five.

It does not control the dirty fraction. The p_{95} column of the same rows moves by a factor of 4.9 at $p = 0.10$ (12.22% down to 2.50%) and a factor of 43.9 at $p = 0.50$ (2.98% down to 0.07%). The knob is set once; what the operator gets from it depends on how many reconcilers happen to be provisioned. This is the sense in which the random oracle loses the point of a critical dirty fraction: p is not expressed in the units of the quantity it is meant to bound, and no setting of it names a consistency level.

But it is still a knob. Inconsistency falls monotonically in p at every R , and monotonically in R at every p . Under EQUILIBRIUM the in-flight population is held constant (Section 5.1.1), so raising p replaces fast BASE transactions with slow ACID ones, the admission rate drops, and less dirtying work enters the system. The random oracle therefore does control inconsistency — indirectly, through the ACID share, rather than by aiming at a target. That is what makes the two oracles comparable at all, and Section 5.5.2 uses it.

5. Experiments

Table 5.8.: X3: results by ACID probability p and reconciler count R . Columns are those of Table 5.3: ϕ_{time} is the share of the run spent in ACID mode, ϕ_{comp} the share of completed transactions that ran under ACID, and *Switches* how often the oracle changed the mode.

R	p	Thr. (tps)	ϕ_{time}	ϕ_{comp}	Switches	p_{50} (%)	p_{95} (%)
1	0.05	0.0904	0.5988	0.0445	438.0	9.42	16.73
	0.10	0.0760	0.6621	0.0934	698.6	6.96	12.22
	0.15	0.0726	0.6789	0.1397	918.4	5.67	10.14
	0.25	0.0733	0.7182	0.2400	1374.0	4.00	7.13
	0.50	0.0831	0.7890	0.4974	2090.8	1.02	2.98
5	0.05	0.3918	0.1288	0.0484	1857.4	8.60	14.90
	0.10	0.3317	0.2142	0.0987	2991.8	5.37	10.50
	0.15	0.2731	0.3045	0.1471	3444.2	1.94	7.80
	0.25	0.1711	0.4980	0.2456	3198.4	0.05	1.95
	0.50	0.0849	0.7873	0.4971	2135.8	0.00	0.18
10	0.05	0.6684	0.0622	0.0483	3123.8	6.73	12.86
	0.10	0.4254	0.1549	0.0988	3828.0	0.36	6.42
	0.15	0.2839	0.2829	0.1475	3590.8	0.05	1.53
	0.25	0.1707	0.4858	0.2451	3187.8	0.00	0.29
	0.50	0.0850	0.7761	0.4969	2137.6	0.00	0.09
15	0.05	0.7920	0.0548	0.0487	3706.2	2.22	12.73
	0.10	0.4416	0.1143	0.0992	3973.4	0.09	2.50
	0.15	0.2942	0.1961	0.1480	3728.6	0.02	0.43
	0.25	0.1750	0.3594	0.2454	3269.8	0.00	0.22
	0.50	0.0876	0.6601	0.4972	2202.0	0.00	0.07

Source. random_p_sweep/random_p_sweep_seed{0..4}.csv, 100 runs, mean over 5 seeds. Seed CV on p_{95} : 5.52% median, 19.02% max.

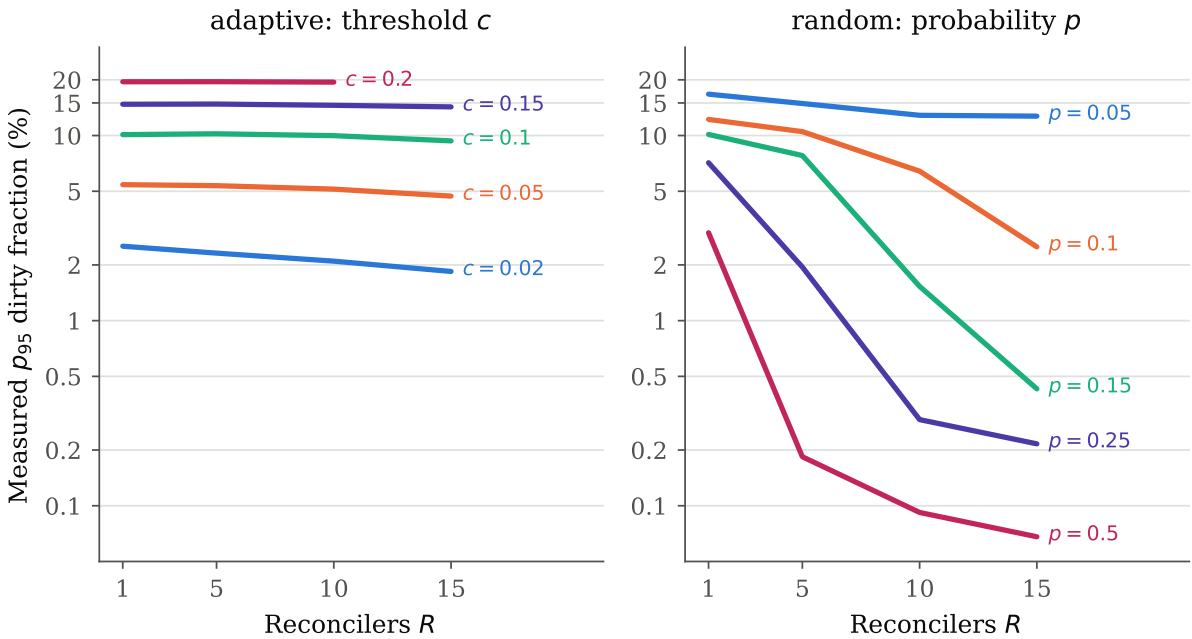


Figure 5.7.: Measured p_{95} dirty fraction against reconciler count, one line per knob setting, on a shared logarithmic scale. Each oracle holds fixed the quantity it reads: the adaptive lines are flat because the threshold is a target, and the random lines fan out because the probability is not.

Figure 5.7 puts the two side by side, and the adaptive panel is the mirror image of the random one. Across the same range of R , the measured p_{95} under the adaptive oracle stays within 0.74 pp of the configured c (Section 5.4.1) while its realised ACID share is what moves — ϕ_{comp} spans 0.108 to 0.472 at $c = 0.02$, a factor of 4.4. Each oracle pins the quantity it reads and lets the other one float. RQ3 is asked in the units of the dirty fraction, so which quantity gets pinned is the whole of the difference.

Two columns to read with care. ϕ_{time} means here exactly what it means in X1 and X2. Both oracles accrue it with the same routine, which attributes the interval between two consecutive decisions to whichever mode was in force across it, so in both cases the column is the share of simulated time the admission mode was ACID, and in both cases dividing it by the number of ACID entries gives the mean length of a stay. What differs is how the share is assembled. The random oracle changes mode 438 to 3973 times per run against 36 to 509 for the adaptive one, so it collects its ACID time in many short visits rather than a few long ones: mean stays of 1.5 to 137s, against 61 to 1835s under the adaptive oracle over the same R .

The column should not be read against p . A probability of 0.05 says that one transaction in twenty is routed to ACID, not that one twentieth of the clock is spent there; ACID work is slow, so the few transactions that get it hold the system in ACID for a large share of the run. That is the same gap between ϕ_{time} and ϕ_{comp} described in Section 5.1.3, and it is widest here, at 0.60 against $\phi_{\text{comp}} = 0.045$ for $p = 0.05$ at $R = 1$.

Throughput at $R = 1$ is also not monotone in p : it falls from 0.0904 to 0.0726 and then rises again to 0.0831, while at $R = 5, 10$ and 15 it falls steadily as p rises. The reason is that R splits a fixed pool rather than adding to it (Section 5.3). Every reconciler is taken out of the 30 foreground processors, so $R = 1$ leaves 29 running transactions and one repairing, and $R = 15$ leaves 15 of each. The two ends of the range are therefore limited by opposite things.

At $R = 1$ there is ample capacity to admit and execute work and almost none to clean up after it: the dirty set drains at $0.5R = 0.5$ resources per second (Section 5.9) against a standing backlog of over a thousand. An ACID transaction that touches a dirty resource waits, and that wait — not the choice of mode — sets the pace of the run. Moving work between the modes cannot speed up something limited by waiting on repair, which is why p has so little purchase here. At $R = 15$ the dirty set drains fifteen times faster and waits are short, but half the pool is now repairing instead of executing, so what limits the run is the capacity left to process transactions at all — and the mode mix, which sets how expensive each transaction is, is exactly what moves that. Throughput therefore falls monotonically in p at the top of the range and not at the bottom. The same mechanism produces the adaptive throughput valley of X1 (Section 5.3.2), with c rather than p setting the mix.

5.5.2. Throughput at matched inconsistency

RQ3 asks whether reading the metrics beats a coin flip *on the same throughput/inconsistency trade-off*. The difficulty is that the two oracles are configured in different terms. Setting $c = 0.10$ asks the adaptive oracle to hold dirtiness near 10%; setting $p = 0.10$ tells the random

oracle to send one transaction in ten to ACID. Those are not the same instruction, and no pair of settings makes the two runs comparable.

What is comparable is what the runs produced. Every run reports both a throughput and the dirtiness it actually reached, so the question can be put on the output side: at the dirtiness this adaptive run reached, how fast was the random oracle? Answering it needs a random run at that same dirtiness, and the random arm was swept at only five probabilities. The five seeds of each configuration are averaged first, so each setting contributes one throughput and one dirtiness.

Locating each adaptive run among the random ones. Take the adaptive run at $R = 5$, $c = 0.10$, which ended up 10.22% dirty at 0.345 tps. The five random runs at $R = 5$ ended up 14.90, 10.50, 7.80, 1.95 and 0.18% dirty. None of them is at 10.22%, but two of them sit either side of it — the run at 10.50% and the run at 7.80% — and the speed at both is known. Table 5.9 does this for every adaptive run: it records where the run landed, which two random runs it fell between, and how close it came to the nearer of the two. That last figure matters because a straight-line estimate is exact at its endpoints and least reliable midway between them: an adaptive run that lands almost on top of a measured random run barely needs the estimate at all. It is reported as a ratio — the adaptive run at 5.43% sits between runs at 2.98 and 7.13%, and $7.13/5.43 = 1.31$, so it is 31% away from the nearer of the two. A value of 1.00 means the two coincide.

Thirteen of the 19 engaged adaptive runs fall between two random runs and can be compared. The remaining six have nothing to sit between — five are dirtier than every random run at that reconciler count, and one, at $R = 1$, $c = 0.02$, is cleaner than every one — and are dropped rather than extrapolated.

Reading the speed in between. For the thirteen that can be compared, the random oracle's speed at the adaptive run's dirtiness is estimated between the two runs on either side. Continuing the example above: the random run at 10.50% dirty managed 0.332 tps and the one at 7.80% managed 0.273 tps, and 10.22 lies nine tenths of the way from 7.80 up to 10.50, so the estimate is nine tenths of the way from 0.273 up to 0.332 — 0.326 tps. The adaptive run's 0.345 against that is a ratio of 1.06, the same throughput to within a few percent. Table 5.10 is that calculation for all thirteen.

How far such an estimate can be trusted depends on the *To nearest* column of Table 5.9. In the example the adaptive run at 10.22% sat 3% below the measured run at 10.50%, so the estimate hardly moves off a measured value and the comparison is close to a direct one. Where the nearest measured run is half again as far away, the estimate is carrying the answer, and the row settles nothing.

Where the comparison is nearly a measurement, the two are equal. Four adaptive runs land within 15% of a measured random run, and all four give the same throughput as the coin flip: 1.00, 1.02, 1.06 and 1.00. These are the rows where almost nothing rests on the

Table 5.9.: Where each adaptive run falls among the random runs at the same reconciler count. *Between* gives the dirtiness of the two random runs on either side; the full set of random runs is the p_{95} column of Table 5.8. *To nearest* is how far the adaptive run sits from the closer of those two, as a ratio — 1.00 means it landed on top of a measured run. A run with no random run on one side cannot be compared and is dropped rather than extrapolated.

R	c	adaptive p_{95} (%)	Between (%)	To nearest
1	0.02	2.52	<i>cleaner than every random run</i>	—
1	0.05	5.43	2.98 – 7.13	1.31×
1	0.10	10.12	7.13 – 10.14	1.00×
1	0.15	14.76	12.22 – 16.73	1.13×
1	0.20	19.52	<i>dirtier than every random run</i>	—
5	0.02	2.32	1.95 – 7.80	1.19×
5	0.05	5.36	1.95 – 7.80	1.46×
5	0.10	10.22	7.80 – 10.50	1.03×
5	0.15	14.79	10.50 – 14.90	1.01×
5	0.20	19.54	<i>dirtier than every random run</i>	—
10	0.02	2.10	1.53 – 6.42	1.37×
10	0.05	5.14	1.53 – 6.42	1.25×
10	0.10	9.98	6.42 – 12.86	1.29×
10	0.15	14.56	<i>dirtier than every random run</i>	—
10	0.20	19.44	<i>dirtier than every random run</i>	—
15	0.02	1.84	0.43 – 2.50	1.36×
15	0.05	4.71	2.50 – 12.73	1.88×
15	0.10	9.36	2.50 – 12.73	1.36×
15	0.15	14.28	<i>dirtier than every random run</i>	—

Source. `random_p_sweep/random_p_sweep_seed{0..4}.csv` and `critical_sweep_seed{0..4}.csv`, means over 5 seeds. $R = 15$, $c = 0.20$ is degenerate and omitted (Section 5.4).

estimate, and they are unanimous. All four are at $c \geq 0.10$, because that is where the random sweep happens to have runs nearby.

The other rows point in no direction. The remaining nine adaptive runs land too far from any measured random run for a single row to be worth much on its own. Taken together they say nothing either: five come out above parity and four below it, the median is 1.05, and no setting of c or of the reconciler count turns one oracle into the reliable winner. Whatever is moving those numbers, it is not a benefit belonging to either oracle. A finer grid of probabilities would let these rows be read one by one rather than only in bulk, and is left to future work (Section 6.3).

The answer to RQ3. Reading the mode from the system’s own metrics does not beat a coin flip in any way this experiment can establish. Where the two oracles can be compared against

Table 5.10.: Throughput of the two oracles at the same measured inconsistency, for the thirteen adaptive runs that fall between two random runs. *Ratio* is adaptive divided by random. Rows are ordered by *To nearest* (Table 5.9), so the comparisons that are closest to a straight measurement come first; the rule marks where the adaptive run stops sitting within 15% of a measured one.

<i>R</i>	<i>c</i>	matched <i>p</i> ₉₅ (%)	Thr. (tps)		Ratio	To nearest
			adaptive	random		
1	0.10	10.1	0.073	0.073	1.00	1.00×
5	0.15	14.8	0.397	0.390	1.02	1.01×
5	0.10	10.2	0.345	0.326	1.06	1.03×
1	0.15	14.8	0.084	0.084	1.00	1.13×
5	0.02	2.3	0.255	0.178	1.44	1.19×
10	0.05	5.1	0.486	0.388	1.25	1.25×
10	0.10	10.0	0.588	0.560	1.05	1.29×
1	0.05	5.4	0.072	0.077	0.93	1.31×
15	0.02	1.8	0.379	0.395	0.96	1.36×
15	0.10	9.4	0.594	0.676	0.88	1.36×
10	0.02	2.1	0.395	0.300	1.32	1.37×
5	0.05	5.4	0.303	0.231	1.31	1.46×
15	0.05	4.7	0.477	0.517	0.92	1.88×

Source. Same two sources as Table 5.9, means over 5 seeds.

a nearby measurement they are equal, and everywhere else the sweep is too coarse to say. A finer grid of probabilities would settle the remaining rows one way or the other, and is left to future work.

None of this is the setting in which feedback would be expected to prove itself. Every run in this thesis is executed under a stationary load, and a controller that adapts has nothing to adapt to when the operating point never moves. Testing whether the feedback loop earns its keep needs a workload that changes while the run is in progress — the terminal-driven mode the simulator already supports, left to future work (Section 6.3). Until that is run, the case for reading the signal is the one made in Section 5.5.1, which does not rest on throughput at all.

5.5.3. Findings

O9 Each oracle controls the quantity it reads, and only that one. The random oracle delivers the ACID share it was told to deliver, whatever the system is doing, but the inconsistency that results swings widely with the reconciler count. The adaptive oracle is the mirror image: it holds the inconsistency where it was asked to and lets the ACID share fall where it may. From this we can understand that neither oracle controls both quantities, and that choosing between them is a choice of which one the operator wants to be able to name.

O10 Neither oracle is measurably faster than the other under a stationary load. Where an adaptive run lands close enough to a measured random run for the comparison to be nearly direct, the two turn in the same throughput; where the data hints otherwise, the random sweep is too coarse to support the hint. From this we can understand that the feedback loop costs nothing in throughput, and that this workload cannot show whether it is worth anything either: with the operating point held constant there is nothing for an adaptive controller to adapt to. A varying workload is the test that would settle it.

5.6. X4: Semantic overlap degree sweep

Semantic overlap is the parameter Zacouris and Acosta [6] use to describe how tightly the models of a multi-model system are tied to one another (Section 2.2.1). Two models overlap semantically when they hold resources standing for the same underlying fact, and a consistency preservation rule ties those resources together. The overlap degree σ (`semanticOverlap`) is the fraction of the resources in the system that carry such a cross-model rule. The simulator draws that set once at start-up, from a seeded draw, and keeps it fixed for the whole run: a write to a resource in the set spawns a bookkeeper transaction against the resource it overlaps in another model, and leaves that resource dirty until a reconciler clears it, while a write to any other resource stays inside its own model and dirties nothing. So σ is a property of the schema, not of the load — it says how much of the rest of the system one write disturbs.

The workload of this sweep is the one used in X1 to X3, unchanged: the same arrival rate, the same transaction size $L = 50$, the same resource count. Only σ moves. That is what makes the sweep worth running on its own: it changes the rate at which the dirty set fills — the plant the oracle regulates — without changing how much work is offered to the system. The budget of X2 was established at a single value of σ , and an operator needs to know over what range of overlap it still holds.

The sweep covers $\sigma \in \{0.00, 0.05, 0.10, 0.20, 0.30, 0.50, 1.00\}$ against $R \in \{1, 5, 10, 15\}$ over five seeds, 140 runs, under HYBRID_ADAPTIVE at the X1 baseline thresholds $c = 0.10$, $r = 0.05$. The $\sigma = 0.30$ block is the shared baseline of every other experiment and reproduces the X1 rows exactly, which is the cross-check that the two sweeps agree where they overlap.

5.6.1. Reading the sweep

Table 5.11 is read downwards in blocks. Each block holds σ fixed and steps through the reconciler count, so a column read across blocks shows what overlap does at fixed repair capacity, and a column read inside one block shows what repair capacity does at fixed overlap. Three things have to be separated when reading it. Whether the oracle fires at all is the switch count and the “deg.” marker: a degenerate cell is pure BASE under a different label, and averaging over it would mix two systems. Whether the oracle meets its budget is p_{95} against the configured $c = 10\%$. What that costs is the throughput and abort columns. The three move independently, and the sweep is informative because each is decided by a different part of the system.

Table 5.11.: X4: dirty-fraction percentiles, oracle behaviour, throughput and abort rate against the semantic overlap degree σ , by reconciler count R . “deg.” marks a cell in which the oracle never switches, so the row is pure BASE under a different label. Abort rate is `aborted_total` over `admitted-plus-aborted`.

σ	R	p_{50} (%)	p_{95} (%)	Switches	ϕ_{time}	ϕ_{comp}	Thr. (tps)	Aborts
0.00	1	0.00	0.00	deg.	0.0000	0.0000	3.8685	0.000
0.00	5	0.00	0.00	deg.	0.0000	0.0000	3.3344	0.000
0.00	10	0.00	0.00	deg.	0.0000	0.0000	2.6681	0.000
0.00	15	0.00	0.00	deg.	0.0000	0.0000	2.0013	0.000
0.05	1	4.50	4.62	deg.	0.0000	0.0000	3.8685	0.000
0.05	5	2.14	2.44	deg.	0.0000	0.0000	3.3344	0.000
0.05	10	0.08	0.22	deg.	0.0000	0.0000	2.6681	0.000
0.05	15	0.04	0.15	deg.	0.0000	0.0000	2.0013	0.000
0.10	1	9.50	9.62	deg.	0.0000	0.0000	3.8685	0.000
0.10	5	7.16	7.47	deg.	0.0000	0.0000	3.3344	0.000
0.10	10	3.20	3.66	deg.	0.0000	0.0000	2.6681	0.000
0.10	15	0.14	0.35	deg.	0.0000	0.0000	2.0013	0.000
0.20	1	6.54	9.90	82	0.6746	0.0601	0.1425	0.025
0.20	5	5.02	9.92	218	0.3856	0.0670	0.6495	0.036
0.20	10	2.02	9.60	195	0.1951	0.0526	0.9897	0.030
0.20	15	7.54	8.22	deg.	0.0000	0.0000	2.0013	0.000
0.30	1	6.80	10.12	80	0.7776	0.0918	0.0726	0.070
0.30	5	5.67	10.22	234	0.5068	0.0966	0.3453	0.074
0.30	10	3.22	9.98	224	0.2886	0.0694	0.5883	0.074
0.30	15	0.43	9.36	162	0.1667	0.0716	0.5941	0.052
0.50	1	7.22	10.26	68	0.8767	0.1384	0.0299	0.154
0.50	5	6.42	10.40	213	0.6423	0.1314	0.1421	0.132
0.50	10	4.95	10.21	240	0.4352	0.0945	0.2581	0.146
0.50	15	2.40	9.95	192	0.3023	0.0773	0.3127	0.137
1.00	1	9.50	22.48	19	0.9784	0.2900	0.0069	0.252
1.00	5	9.49	12.14	78	0.8836	0.2351	0.0274	0.337
1.00	10	7.46	11.34	133	0.7830	0.1464	0.0446	0.338
1.00	15	5.73	10.79	159	0.6802	0.1042	0.0607	0.282

Source. `semantic_overlap_sweep/semantic_overlap_sweep_seed{0..4}.csv`, 140 runs, mean over 5 seeds.

Seed CV on p_{95} : 0.41% median, 7.21% maximum.

Engagement has a threshold in σ . Thirteen of the 28 cells are degenerate, and they are not scattered: every cell at $\sigma \leq 0.10$ degenerates at every R , plus $\sigma = 0.20$ at $R = 15$. This is the degeneracy rule of finding O6 (Section 5.4.2) again, read along a different axis. Coupling density sets the BASE steady-state dirty fraction, and the oracle engages only where that steady state reaches c . At $\sigma = 0.10$, BASE alone runs at $p_{95} = 9.62\%$ at $R = 1$, already below $c = 0.10$ and falling to 0.35% at $R = 15$; at $\sigma = 0.20$ it clears c at $R \leq 10$ and not at $R = 15$. The boundary is sharp: between $\sigma = 0.10$ and $\sigma = 0.20$ the sweep goes from zero switches in twenty runs to 82–218 switches per run. The practical form of this is that a hybrid deployment on a schema whose models barely overlap is pure BASE regardless of configuration.

Where it engages, the budget does not move with overlap. Once the oracle is firing, more overlap does not make the system more inconsistent. It makes the oracle work harder, and the tail comes out where it was configured. The eleven engaged cells at $\sigma \in \{0.20, 0.30, 0.50\}$ all land within 0.64 pp of the configured $c = 10\%$, and neither σ nor R moves the result in any direction inside that band. This is the strongest form the X2 claim takes anywhere in this chapter: the threshold is a budget on the realised tail, and how much the models overlap does not enter it.

What σ does control is the price. What rises with overlap is not the inconsistency but the cost of keeping it inside the budget. At $R = 1$ throughput falls steadily — at $\sigma = 0.20$: 0.14 tps; at $\sigma = 0.30$: 0.073 tps; at $\sigma = 0.50$: 0.030 tps; at $\sigma = 1.00$: 0.0069 tps — a factor of 21 across the engaged range, and a factor of 560 against the 3.87 tps the same R delivers where the oracle never fires. Aborts climb with it, from 0.025 to 0.25, while the tail itself moves by 0.36 pp. Holding the budget fixed as overlap rises is paid for entirely in throughput and aborts.

One cell breaks the invariance, and it is the interesting one. At $\sigma = 1.00$, $R = 1$ the tail reaches 22.48%, twice the budget and the only such miss in this chapter. It is not a control failure, and the entry measurements say so: the oracle switches to ACID at a dirty fraction of 10.5% on average, on the line it was given, and everything above that arrives after the brake is already on. A switch changes the mode of transactions admitted after it, not of the thirty already in flight, and at $\sigma = 1.00$ what those thirty still have to deposit is enormous. Over the run, 244 BASE root transactions produce 25 397 dirty-mark events — about 104 resources from a single transaction, some 2% of the whole system — so thirty of them in flight carry on the order of 3000 marks already committed to happen, against a threshold that sits at 500 resources. The threshold is crossed by a fraction of that output and the rest lands regardless. What decides the tail is then how long the overshoot takes to unwind: the single reconciler clears 25 000 resources over the run, exactly its 0.5 s^{-1} flat out from start to finish, and the mean excursion lasts 5239 s. Reconcilers do not prevent the overshoot — the peak is still 18.5% at $R = 15$ — but they drain it fifteen times faster, and p_{95} comes back to 10.79%. The limit is capacity, not the threshold.

5.6.2. Findings

- O11 How much the models overlap decides whether the oracle has anything to do, not how well it does it.** Below a threshold in σ the run is pure BASE at every reconciler count; above it, the tail lands within 0.64 pp of the configured budget whatever the overlap. From this we can understand that σ is not a knob that trades consistency against throughput: what it changes is the price of the budget, which rises by a factor of 21 in throughput across the engaged range. An operator can carry one value of c across schemas that overlap to different degrees, but not one throughput expectation.
- O12 The budget holds only while reconciliation keeps up with the dirtying it has to repair.** At $\sigma = 1.00$, $R = 1$ the oracle runs almost entirely in ACID and still overshoots c by a factor of two, and the miss disappears as soon as R rises. From this we can understand that the threshold is a conditional guarantee: once repair is slower than the dirtying, the oracle has already spent everything the policy allows, and no setting of c or r recovers it. The remedy there is provisioning, not tuning.

5.7. X5: Resources per transaction sweep

L (`maxTransactionResources`) is the largest number of resources a single transaction may touch. Each transaction draws its size uniformly from 1 to L , draws a model count uniformly from one to four (`maxTransactionModels`, a configured baseline rather than a property of the simulator, held at 4 by every experiment in this chapter), and splits its resources over that many of the ten models, taking a contiguous block inside each. That cap says how many models a transaction reaches into, not how much of them it holds: at the baseline $L = 50$ the block taken from one model averages ten of the 500 resources that model owns. Nothing else moves with L . The schema is the one every other experiment uses, ten models and 5000 resources at $\sigma = 0.30$, so how far a single write propagates is unchanged; the workload is the one every other experiment uses, EQUILIBRIUM with 30 transactions in flight, so the amount of work offered to the system is unchanged too. What L decides is only how deep one transaction reaches into that fixed schema.

An example fixes the range. At the baseline $L = 50$ a transaction touches 25.5 resources on average out of the 5000 in the system, each under its own lock until it commits, so the 30 transactions in flight hold some 750 between them, 15% of the data set. At $L = 400$ the same draw averages 200.5, and the 30 in flight demand around 6000 — more claims than the system has locks to give. Nothing about the schema or about the arrival of work differs between those two settings; each transaction simply reaches deeper.

That makes this sweep a check rather than a new question. X2 established the budget at one operating point and X4 showed it survives a change of schema; here the axis is how deep a transaction goes, and the two things worth measuring are whether the tail still lands at c and, if it does, which measured quantity absorbed the change instead. The sweep covers $L \in \{5, 10, 25, 50, 100, 200, 400\}$ against $R \in \{1, 5, 10, 15\}$ over five seeds, 140 runs, under HYBRID_ADAPTIVE at $c = 0.10$, $r = 0.05$ and $\sigma = 0.30$. $L = 50$ is the baseline shared with every

other experiment.

Table 5.12 is arranged for that pair of questions: one block per reconciler count, rows ascending in L , so each block reads as one repair capacity taken from small transactions to large ones. The switch count says whether the oracle is engaged at all, p_{95} against the configured $c = 10\%$ is the quantity under control, p_{99} says how far the excursions above it reach, and the last three columns are what holding the budget costs. Two of those need a word before the table. *Lat.* is the mean time from creation to commit (Section 5.1.3), which is what grows when a transaction has more resources to process. *Aborts* is aborted_total over admitted-plus-aborted, the share of started transactions that did not finish.

Table 5.12.: X5: what moves and what does not as the resources per transaction L grow, by reconciler count R .

L	R	p_{50} (%)	p_{95} (%)	p_{99} (%)	Switches	Thr. (tps)	Lat. (s)	Aborts
5	1	7.35	9.74	9.98	111	0.6377	18.0	0.004
10	1	7.34	9.80	10.02	105	0.3458	40.5	0.012
25	1	7.12	9.96	10.27	92	0.1450	127.8	0.046
50	1	6.80	10.12	10.68	80	0.0726	261.8	0.070
100	1	6.46	10.29	11.92	66	0.0358	497.1	0.106
200	1	6.22	10.78	17.38	54	0.0177	820.2	0.283
400	1	5.91	11.87	21.46	43	0.0091	1221.3	0.522
5	5	7.08	9.77	10.02	457	3.0352	4.2	0.001
10	5	6.95	9.84	10.13	430	1.6682	9.2	0.006
25	5	6.44	10.07	10.46	325	0.6994	28.6	0.027
50	5	5.67	10.22	10.82	234	0.3453	67.2	0.074
100	5	4.00	10.19	11.03	147	0.1548	156.7	0.208
200	5	2.29	10.02	11.38	86	0.0639	334.0	0.403
400	5	0.94	9.48	11.95	45	0.0254	538.4	0.646
5	10	6.01	9.67	10.00	615	5.4951	3.1	0.000
10	10	5.29	9.72	10.13	522	2.9549	6.3	0.001
25	10	4.03	9.86	10.46	319	1.1752	16.7	0.016
50	10	3.22	9.98	10.82	224	0.5883	37.4	0.074
100	10	1.23	9.77	10.88	134	0.2447	98.2	0.195
200	10	0.58	9.14	10.70	69	0.0975	224.8	0.359
400	10	0.42	8.44	10.42	39	0.0439	369.9	0.532
5	15	5.14	9.60	9.94	493	7.5079	2.7	0.000
10	15	3.42	9.52	10.00	451	3.6856	6.1	0.000
25	15	0.83	9.44	10.16	254	1.2084	18.4	0.006
50	15	0.43	9.36	10.37	162	0.5941	38.2	0.052
100	15	0.42	9.22	10.59	113	0.2937	81.7	0.172
200	15	0.32	8.64	10.08	58	0.1294	181.0	0.318
400	15	0.52	7.91	9.70	31	0.0676	262.8	0.426

Source. max_txn_resources_sweep/max_txn_resources_sweep_seed{0..4}.csv, 140 runs, mean over 5 seeds.

Seed CV on p_{95} : 0.26% median, 4.23% maximum.

5.7.1. Reading the sweep

Every cell switches, between 31 and 615 times, so no cell is degenerate and all 28 are comparable. Down each block the p_{95} column hardly moves. Over the 24 cells with $L \leq 200$ it spans 8.64% to 10.78% against the configured $c = 10\%$, and over $L \in [5, 100]$, where the resources per transaction grow by a factor of 20, it spans 9.22% to 10.29% — within 0.78 pp of the configured value. So L , which decides how many resources a single transaction can dirty and therefore how fast the dirty set fills, does not move the level at which that set is held.

What moves is in the cost columns, and neither of the two is the oracle's doing. A transaction takes proportionally longer the more resources it touches, because the simulator charges processing time per resource: at $R = 1$, at $L = 5$: 18.0 s from creation to commit; at $L = 400$: 1221 s. With 30 transactions held in flight throughout, ones that take longer individually complete less often, and that is the whole of the throughput column: at $R = 1$, at $L = 5$: 0.638 transactions per second; at $L = 400$: 0.0091. The second cost is the abort rate, which rises with L in every block: at $R = 10$, at $L = 5$: 0.000; at $L = 50$: 0.074; at $L = 200$: 0.359; at $L = 400$: 0.532. Larger transactions hold more locks at once, so they collide more often and more of the work already done is thrown away.

Reconciliation is the one place where this outcome is worth an argument, because the obvious expectation runs the other way: a transaction that touches more resources dirties more of them, and should therefore load the reconciler more. Per transaction it does, exactly in proportion. Dividing the `dirty_mark_events_total` of each run by the transactions it completed gives, at $R = 1$, at $L = 5$: 0.8 dirty marks per transaction; at $L = 400$: 53. What breaks the inference is that the sweep holds 30 transactions in flight rather than a fixed arrival rate, so the larger ones run proportionally slower and the marks arrive no faster: at $R = 1$, at $L = 5$: 0.509 marks per second; at $L = 400$: 0.486. The served side agrees, and at high R it moves against the expectation rather than merely failing to confirm it: the clear rate of Section 5.1.3 against its ceiling of $0.5R$ gives, at $R = 15$, at $L = 5$: 7.45 resources cleared per second of a possible 7.5, a saturated pool; at $L = 400$: 4.43, three fifths of capacity. Larger transactions give the reconciler less to do, not more, because the lock conflicts they cause hold down the rate at which resources are touched at all.

Two smaller patterns come with it. The median dirty fraction falls while the tail stays where it was configured: at $R = 15$, at $L = 5$: 5.14%; at $L = 400$: 0.52%, against a p_{95} that stays between 7.91% and 9.60% throughout. With larger transactions the dirty set sits lower and spikes harder, which is also what the falling switch counts show. And the excursions above the budget grow only where repair is scarce: p_{99} at $R = 1$, at $L = 5$: 9.98%; at $L = 400$: 21.46%, while at $R \geq 10$ it stays near 10% at every L . That is O12 read on this axis rather than on the overlap axis.

The $L = 400$ block sits outside the regime the rest of the sweep measures. The 30 transactions in flight there demand about 6000 resources against the 5000 locks the system has, so lock conflict rather than the oracle sets the admission rate: the dirty set never gets the chance to fill, and p_{95} undershoots: at $R = 10$: 8.44%; at $R = 15$: 7.91%. The block is a stress anchor, not a like-for-like scaling point, and is excluded from the invariance claim above.

5.7.2. Findings

- O13 How many resources a transaction touches does not move the tail.** As L grows by a factor of 20 the measured p_{95} stays within 0.78 pp of the configured c at every reconciler count, with no degenerate cell. From this we can understand that O11 was not an artefact of the axis it was measured on: the budget holds on both workload dimensions, the one that sets how far a write spreads and the one that sets how much a transaction touches.
- O14 What L costs is latency and aborts, not reconciler load.** Larger transactions take proportionally longer to process and collide more often — at $R = 1$, at $L = 5$: 18.0 s per transaction and an abort rate of 0.004; at $L = 400$: 1221 s and 0.52 — while the dirty marks the reconciler has to clear arrive no faster, at $L = 5$: 0.509 per second; at $L = 400$: 0.486. From this we can understand that the two workload axes are priced differently: semantic overlap loads the reconciler and so raises the cost of the budget directly, while L is paid for in the foreground, in waiting and in wasted work. An operator with a throughput problem should look at semantic overlap first, and at L only if aborts are high.

5.8. Ablation: the smoothing factor and its removal

The five sweeps above evaluate one system. This section does not: it reports the measurement that produced that system, by sweeping a parameter the evaluated oracle no longer has. It is placed here rather than in the approach chapter because the argument for the removal is empirical, and placed after X5 rather than among the numbered sweeps because α is not a knob any result in this chapter can be reproduced with.

The oracle evaluated in this thesis compares its thresholds against the dirty fraction measured at the instant of the decision. An earlier version of the implementation did not: it compared them against an exponentially weighted moving average of that fraction, with a smoothing factor α setting how much of each new sample entered the estimate. The ablation serves two purposes. It establishes that the bound reported in X2 is a property of the design rather than of a filter setting, and it is the measurement on which the decision to remove the filter rests.

The sweep covers $\alpha \in \{0.0, 0.1, \dots, 1.0\}$ at $R \in \{1, 5, 10, 15\}$ across three seeds, 132 runs. The endpoints are included deliberately: $\alpha = 0$ freezes the estimate and $\alpha = 1$ removes the smoothing entirely. The $\alpha = 1$ rows therefore describe the oracle as it is evaluated in the rest of this chapter, and the remaining rows describe what the filter was doing before it was taken out.

Two caveats attach to the numbers below. They were collected on three seeds rather than five, and against the version of the code that still carried the filter, so they are not directly comparable digit-for-digit with the five-seed tables above — the $\alpha = 1$ column is the like-for-like anchor, and it agrees with the X1 and X2 rows to within the seed dispersion. The

confirmation that matters does not rest on this sweep alone: X1 and X2 were re-run after the removal, and the change they show is reported in Section 5.8.5.

Table 5.13.: Ablation: dirty-fraction and oracle-behaviour statistics by α and R . The $\alpha = 0$ row is the degenerate case, included as a convergence check against pure BASE. Intermediate values of α are omitted here; the full grid is in the appendix.

R	α	p_{50} (%)	p_{95} (%)	Switches	ϕ_{time}	ϕ_{comp}	Thr. (tps)
1	0.0	29.50	29.65	0.0	0.0000	0.0000	3.8654
1	0.1	5.97	11.11	57.7	0.9041	0.1698	0.0758
1	0.2	6.15	10.59	64.7	0.8654	0.1341	0.0737
1	0.5	6.49	10.22	74.0	0.8204	0.1073	0.0721
1	1.0	6.80	10.13	79.7	0.7866	0.0924	0.0720
5	0.0	27.20	27.53	0.0	0.0000	0.0000	3.3323
5	0.1	2.95	10.71	144.7	0.6656	0.1280	0.2787
5	0.2	4.43	10.53	177.3	0.5970	0.1111	0.3251
5	0.5	5.29	10.33	212.3	0.5491	0.1011	0.3410
5	1.0	5.67	10.25	232.7	0.5034	0.0969	0.3443
10	0.0	23.38	23.94	0.0	0.0000	0.0000	2.6651
10	0.1	0.65	10.11	157.7	0.5723	0.0987	0.4129
10	0.2	1.59	10.04	186.0	0.4391	0.0841	0.4922
10	0.5	2.39	9.99	206.7	0.3265	0.0729	0.5565
10	1.0	3.28	9.97	225.3	0.2920	0.0702	0.5863
15	0.0	17.74	18.46	0.0	0.0000	0.0000	1.9989
15	0.1	0.15	9.42	128.3	0.3947	0.0847	0.4997
15	0.2	0.29	9.39	146.3	0.2940	0.0775	0.5510
15	0.5	0.39	9.40	159.3	0.2032	0.0726	0.5835
15	1.0	0.43	9.34	160.7	0.1656	0.0709	0.5933

Source. experiments-old-ewma-0.2/alpha_sweep/alpha_sweep_seed{0,1,2}.csv, 132 runs, mean over 3 seeds.

5.8.1. The $\alpha = 0$ anchor

The degenerate setting is checked first, because it validates the implementation. At $\alpha = 0$, across all four R values and all three seeds, the switch count, ϕ_{time} and completed_acid are identically zero, not merely close. This is the expected behaviour: with $\alpha = 0$ the smoothed estimate never updates past its first sample, which is taken at an empty dirty set, so the entry threshold is never crossed and the run is pure BASE. The throughput of those rows agrees with the three-seed X1 BASE rows to five significant figures at every R : 3.8654, 3.3323, 2.6651 and 1.9989. (The five-seed BASE rows of Table 5.3 differ from these in the third decimal, which is the two additional seeds and not the oracle — BASE never consults it.) Together these confirm that α reached the oracle rather than being silently ignored, and that the two sweeps agreed where they overlapped.

5.8.2. The bound does not depend on the smoothing

The main result of the sweep is negative in the useful sense: over the whole engaged range of α , from 0.1 to 1.0, the measured p_{95} stays between 9.34% and 11.11% against the configured $c = 10\%$. The entry side of the loop is therefore essentially decoupled from the smoothing parameter, and the tracking result of Section 5.4 is not an artefact of any particular choice of α . This is also what makes removing the filter safe: the guarantee the oracle is built to provide is the one quantity the filter demonstrably does not touch.

The reason is the asymmetry described in Section 5.9. Entry is evaluated in BASE, where samples arrive every one to two simulated seconds and the dirty fraction rises slowly, so even a heavily smoothed estimate crosses c within a fraction of a percentage point of the right moment. How much the estimate is smoothed barely changes that.

5.8.3. What the smoothing changed

Everything the smoothing factor affects sits on the exit side of the loop and in the switching rate. Table 5.14 isolates the change that removing it makes, by placing the earlier default of $\alpha = 0.2$ next to the $\alpha = 1.0$ rows that describe the oracle without a filter.

Table 5.14.: Ablation: the effect of removing the smoothing. $\alpha = 0.2$ is the earlier default, $\alpha = 1.0$ the unfiltered oracle. $\bar{d}_{\text{exit}}$ is the mean live dirty fraction at the instant the oracle returns to BASE — the effective recovery threshold of Section 5.9 — against a configured $r = 5\%$.

R	α	$\bar{d}_{\text{exit}}$ (%)	ϕ_{time}	Thr. (tps)	Switches
1	0.2	2.01	0.8654	0.0737	64.7
1	1.0	3.57	0.7866	0.0720	79.7
5	0.2	0.79	0.5970	0.3251	177.3
5	1.0	3.46	0.5034	0.3443	232.7
10	0.2	0.21	0.4391	0.4922	186.0
10	1.0	2.84	0.2920	0.5863	225.3
15	0.2	0.03	0.2940	0.5510	146.3
15	1.0	1.98	0.1656	0.5933	160.7

Source. experiments-old-ewma-0.2/alpha_sweep/alpha_sweep_seed{0,1,2}.csv, columns exit_dirty_live_mean, acid_fraction_time, throughput_tps and mode_switch_count. Mean over 3 seeds.

The recovery threshold. The filter is what made r meaningless. Under smoothing the oracle leaves ACID with a true dirty fraction of 2.01% at $R = 1$ falling to 0.03% at $R = 15$, against a configured 5%: the faster reconciliation runs, the further past the configured level the system drains before the estimate catches up. Removing the filter moves that exit point to 3.57% and 1.98% respectively, and compresses its spread from $R = 1$ to $R = 15$ from a factor of 67 to a factor of 1.8.

The unfiltered oracle still does not realise r exactly, and the residual gap is the point of Section 5.9: with the filter gone the oracle sees the true dirty fraction, but it only sees it when a transaction arrives at admission, and between two arrivals a fast reconciler pool can carry the backlog well past r . Removing the smoothing eliminates one of the two lags; the sampling interval is the other, and it is the one that remains.

ACID time and throughput. Exiting earlier means spending less of the run in ACID. ϕ_{time} falls at every R , by 9% in relative terms at $R = 1$ and by 44% at $R = 15$, and throughput moves with it: from 0.325 to 0.344 tps at $R = 5$, 0.492 to 0.586 at $R = 10$ and 0.551 to 0.593 at $R = 15$. The one exception is $R = 1$, where throughput is essentially flat and slightly lower, because at that provisioning the oracle is in ACID for the great majority of the run under either setting and the small change in ϕ_{time} has little left to recover. Since the unfiltered oracle also switches more often, these gains rule out mode transitions as a significant direct cost: what drives throughput is time spent in ACID, not the number of times the mode changes.

Switching. This is the one quantity that moves against the removal. Switch counts rise from 64.7 to 79.7 at $R = 1$, 177.3 to 232.7 at $R = 5$, 186.0 to 225.3 at $R = 10$ and 146.3 to 160.7 at $R = 15$ — between 10% and 31% more transitions. Reducing this is the only thing the filter measurably bought. The increase is bounded, though, and the comparison against $\alpha = 0.1$ shows why: the hysteresis band, not the filter, is doing most of the anti-flap work, and the filter was contributing a further reduction on top of it rather than providing the mechanism.

5.8.4. Why the smoothing was removed

The sweep separates the two halves of the oracle. The entry side, which carries the guarantee, is insensitive to the smoothing factor over its entire range. The exit side, which determines how much throughput is given up and whether r names a real operating point, is sensitive to it throughout.

That split is what decided the design. Filtering is worth its cost when a signal carries measurement noise, and the dirty fraction carries none: it is an exact count of a set the simulator maintains, not an estimate of an unobservable quantity. What the filter attenuated was therefore not noise but real movement, and it did so on top of a second lag — the admission-driven sampling interval — that cannot be removed by tuning. Of the two, the filter was the larger contributor on the exit side, and the two together drove the effective recovery threshold two orders of magnitude below its configured value at $R = 15$.

Set against that, the filter bought a 10–31% reduction in the switching rate, on a mechanism the hysteresis band already provides. Removing it leaves the guarantee untouched, returns between a tenth and a half of the ACID time budget depending on provisioning, and makes the recovery threshold mean something much closer to what it says. The oracle described in Section 4.6.3 therefore acts on the instantaneous dirty fraction, and the smoothing factor is no longer a parameter of the system.

5.8.5. Confirmed by re-running the sweeps

The sweep above predicted the removal would be safe. X1 and X2 were then re-run against the filter-free implementation on five seeds, which turns that prediction into a measurement. Table 5.15 places the two sets of HYBRID_ADAPTIVE rows side by side at the baseline $c = 0.10$.

Table 5.15.: Effect of removing the smoothing, measured by re-running X1. “before” is the filtered oracle at the earlier default $\alpha = 0.2$, three seeds; “after” is the filter-free oracle, five seeds. The guarantee (p_{95} against $c = 10\%$) is unchanged; the cost of providing it falls.

R	Thr. (tps)			ϕ_{time}			p_{95} (%)	
	before	after	change	before	after	change	before	after
1	0.0737	0.0726	−1.5%	0.8654	0.7776	−10.1%	10.59	10.12
5	0.3251	0.3453	+6.2%	0.5970	0.5068	−15.1%	10.53	10.22
10	0.4922	0.5883	+19.5%	0.4391	0.2886	−34.3%	10.04	9.98
15	0.5510	0.5941	+7.8%	0.2940	0.1667	−43.3%	9.39	9.36

Source. “before” from experiments-old-ewma-0.2/reconciler_sweep/reconciler_sweep_seed{0,1,2}.csv at $\alpha = 0.2$; “after” from Table 5.3.

The measured outcome is slightly better than the sweep predicted. The p_{95} column moves by at most 0.47 pp and stays inside the X2 tracking bound at every R , so the guarantee is untouched — and across the full X2 grid that bound in fact tightened, from 1.3 pp to 0.74 pp. The ACID time share falls at every R , by 10% at $R = 1$ and 43% at $R = 15$, and throughput rises at every R except the first, by up to 19.5%. The switching rate rises correspondingly, from 65 to 80 switches at $R = 1$ and from 146 to 162 at $R = 15$, which is the cost the hysteresis band was always paying for and which the filter was quietly reducing on top of it.

The largest single effect is the one reported in Section 5.9: the effective recovery threshold. Under the filter the mean live dirty fraction at exit fell from 2.01% at $R = 1$ to 0.03% at $R = 15$; without it, it runs 3.64% to 2.05% against a configured r of 5%. The configured recovery threshold went from describing nothing at high R to describing an upper bound that is approached within 3 pp. Removing the filter therefore did not trade the guarantee for throughput — it returned throughput, ACID budget and the meaning of r at once, which is what should be expected when a filter is applied to a signal that carries no noise.

5.9. The effective recovery threshold

Every hybrid run in this chapter carries a second oracle threshold that no sweep varies: the oracle enters ACID when the dirty fraction reaches the critical threshold c , and returns to BASE once it has fallen back to the recovery threshold r — $r = 0.05$ at the baseline, $r = 0.5c$ throughout X2. The effect reported here is present in all five sweeps, which is why it is placed after them rather than inside one of them. In one sentence: r is an upper bound on where an ACID excursion ends, not the point at which it ends, and runs leave ACID cleaner than they

were asked to — by more, the faster reconciliation is provisioned.

Table 5.16.: Where the oracle actually switches, how long an ACID transaction waits, and how often the oracle looks. Measured under HYBRID_ADAPTIVE at $c = 0.10$, $r = 0.05$. Dirty fractions are percentages of the 5000 resources; the *miss* columns are the distance from the threshold that fired, in percentage points, positive meaning dirtier than configured.

R	Entry (BASE $\rightarrow$ ACID)		Exit (ACID $\rightarrow$ BASE)			Mean ACID wait (s)	Interval between looks (s)	
	mean d	miss vs c	mean d	miss vs r	p_{95}		in ACID	in BASE
1	10.08	+0.08	3.64	-1.36	4.88	392.5	107.2	3.37
5	10.11	+0.11	3.42	-1.58	4.92	70.2	15.0	1.58
10	10.16	+0.16	2.85	-2.15	4.86	32.5	7.0	1.30
15	10.12	+0.12	2.05	-2.95	4.74	22.5	3.9	1.51

Source. `critical_sweep_seed{0..4}.csv`, $c = 0.10$ rows; columns `entry_dirty_live_mean`, `exit_dirty_live_{mean,p95}` and `acid_dirty_block_time_mean_ms`, mean over 5 seeds. The transitions are counted from `entry_dirty_live_count`: 40, 117, 112 and 81 entries respectively. *Mean ACID wait* is the average length of one wait on a repair, per wait rather than per transaction, as in Table 5.7. The intervals are simulated time in each mode divided by the root transactions tagged in that mode; `started_{acid,base}` counts those tags.

What a look is. The last two columns need saying plainly, because the oracle has no timer and the word *interval* would otherwise suggest one. There is exactly one place where the rule is evaluated: a root transaction reaches admission carrying no mode tag, and the routine that gives it one reads the live dirty count and compares it against c and r (Section 4.6.5). One tagged root transaction is one look, and nothing else — not the passing of time, not a bookkeeper transaction, not a reconciliation finishing — causes another. Between two looks the oracle simply does not know what the dirty set is doing. The two columns divide the simulated time spent in each mode by the number of transactions tagged in it, so they report how long the rule went unevaluated on average: every 1.3 to 3.4 seconds in BASE, and every 3.9 to 107 seconds in ACID.

This is the oracle’s own sensor and not the thesis’s measuring instrument. The dirty fractions reported everywhere else in this chapter come from the metrics sampler, which fires once per simulated second in both modes (Section 5.1.3); it is the divergence between the two that this section is about.

Entry is prompt, exit is late. At entry the threshold means what it says: the oracle fires 0.08 to 0.16 pp above the configured 10%, with no trend in R . Exit misses, and always on the clean side. The mean dirty fraction at the return to BASE is 3.64% at $R = 1$, which is 1.36 pp below the configured r of 5%, and 2.05% at $R = 15$, which is 2.95 pp below it; the miss grows steadily with reconciliation capacity. Figure 5.8 is the two edges drawn against the thresholds they were given. The p_{95} column caps the miss: the dirtiest exits sit at 4.74–4.92%, just under r at every R .

5. Experiments

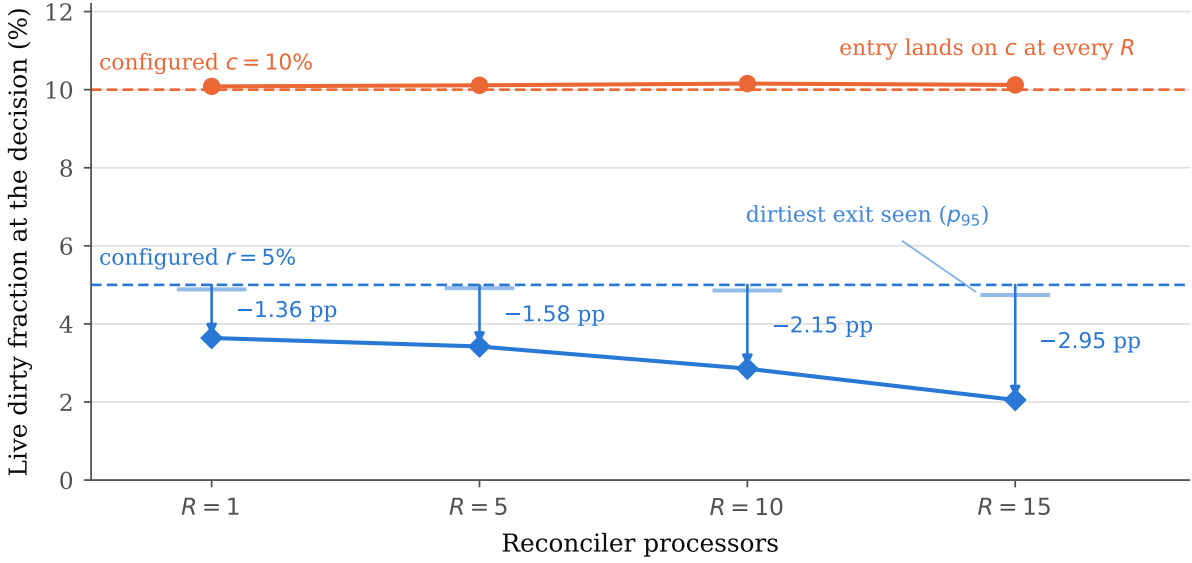


Figure 5.8.: What the thresholds were set to, and where the oracle actually switched, at $c = 0.10$ and $r = 0.05$. Entry lands on c at every reconciler count; exit falls short of r by a distance that grows with reconciliation capacity. The short bars are the p_{95} exit, the dirtiest observed. Means over five seeds, the data of Table 5.16.

Why the exit runs past r . Looks are scarce in ACID because ACID is the mode in which transactions stop arriving. One that needs a resource the reconcilers have not cleared is parked until they do (Section 4.4), it holds its in-flight slot while parked, and under EQUILIBRIUM nothing new is admitted until something completes. The wait column is the size of that stall: one wait on a repair averages 22.5 s at $R = 15$ and 392 s at $R = 1$, against mean excursions of 103 s and 976 s. Hence the 3.9 to 107 s between looks in ACID against 1.3 to 3.4 s in BASE — and a larger budget lengthens the wait further, because it grows with the dirtiness the transaction was admitted into (at $R = 10$, 7.9 s at $c = 0.02$ against 59.0 s at $c = 0.20$, Table 5.7).

The interval is far shorter than the wait, and why it is says what the interval column hides. The thirty in-flight transactions are not all parked at the same moment; the ones that are not still finish and free their slots, so admissions keep trickling in and the average comes out at a few seconds even where a single wait runs to twenty of them. What the average smooths over is the stretches in which enough of the population is parked at once that nothing is admitted at all, and it is those, not the mean, that carry an exit past r . One excursion at $R = 15$, whose mean interval is the 3.9 s of the table, contains a 33-second run in which the oracle is never consulted; the dirty set crosses r inside it and is at 1.66% by the time the next transaction arrives to notice.

Repair meanwhile neither pauses nor draws on the foreground pool: under pure BASE the reconcilers complete exactly $25,000R$ reconciliations for every $R \leq 20$, a drain of $0.5R$ resources per second, against a standing backlog of 394 to 1473 resources that Little’s law matches to the measured recovery latency within 3%. Only at $R = 25$ does the pool outrun

the workload, its backlog collapsing to 2.1 resources, which is what makes $R = 25$ degenerate in X1 (Section 5.3.2). The dirty set therefore keeps draining at a rate set by R while the foreground is stalled and the oracle is not looking, and the excursion ends wherever the drain has got to when some transaction finally reaches admission. That gives the direction of the effect but not its size: the mean interval times the drain rate accounts for the undershoot at $R = 1$ and is too small at $R = 15$, and the distribution of the looks within an excursion, which is what would settle it, is not measured here.

The slack protects the budget rather than weakening it. The coarse sampling does not reach the two edges equally. Entry is decided in BASE, where a look arrives every one to three seconds and dirtiness accumulates slowly under stationary load; exit is decided in ACID, where looks are rare and the quantity moves fastest. Exits are therefore late and never early, and a run ends up cleaner than its configuration implies rather than dirtier, so the tail bound of X2 is unaffected by the slack in r (Section 5.4.1). What it costs is ACID time rather than consistency: excursions run 2.0 to 3.1 times longer than the drain from $cN = 500$ to $rN = 250$ resources requires — 976 s against 500 s at $R = 1$, 103 s against 33 s at $R = 15$. Sampling on a fixed interval of simulated time rather than once per admission would remove most of that, and is left to future work.

5.10. Threats to validity

- **Stationary load only (EQUILIBRIUM).** Adaptive control would be expected to show its clearest advantage when the operating point moves, and constant load cannot test that. The claim is scoped accordingly in Section 4.2, and the X3 claim about the knob (O9) is phrased in terms of transfer across *configurations* rather than adaptation within a run. The stationarity of the load is stated at the head of Section 5.5 as well, because the throughput comparison there is run in the regime least favourable to feedback and should be read in that light.
- **Five seeds per configuration.** Mitigated but not removed by how little the results move between seeds. The random arm of X3 is the high-variance one — 5.52% median seed CV on p_{95} against 0.25% for the adaptive arm — so the O9 conclusion rests on the arm with the larger error bars, in the direction that favours the conclusion. The throughput ratios of Table 5.10 carry seed standard deviations of 0.004 to 0.051, which is small against the 0.82 to 1.39 range of the ratios themselves but is computed from five runs and should be read as an indication rather than an interval. The ablation of Section 5.8 is the exception at three seeds, and its conclusion is independently confirmed at five in Section 5.8.5.
- **The blocking counter is not a transaction count.** `acid_txn_blocked_on_dirty_count` counts distinct root transaction identifiers that blocked at least once, and the set it counts is not a subset of the admitted one — it includes transactions still parked at the end of the run and transactions that aborted while parked. It is therefore reported in this

chapter only as a raw activity total (Table 5.2) and never as a rate against admissions, and no claim rests on it. Measuring how often an ACID transaction blocks, rather than how long it waits, would need a per-transaction instrument.

- **reconciler_busy_fraction is misnamed.** It is not a utilisation but the fraction of samples in which *every* reconciler was busy at the same instant, taken once per simulated second against a service time of two seconds. The phase relationship between the two makes it unreliable: it reads 0.5 at $R = 1$ and 0.0 at $R \geq 10$ under pure BASE, while the same runs sustain exactly $0.5R$ completed reconciliations per second against a standing queue of over a thousand resources — a pool that is in fact fully saturated. Saturation is established in this chapter from the completion rate and the queue depth instead (Section 5.9); no result uses this column, and it should not be read from the exported CSVs.
- **$R = 0$ is a different mechanism, not zero capacity.** At $R = 0$ the CPR work is performed inline by the admitting transaction, so the run has no reconciliation queue and 1.16M mark events against 26k at $R = 1$. The $R = 0$ rows are a reference line and are not points on the same capacity axis as $R \geq 1$.
- **Simulated rather than real execution, and a single workload shape.** The absolute numbers are properties of the simulator; the comparisons between configurations are the transferable part.
- **The matched comparison of X3 reads between measured points.** The random arm is sampled at five values of p , and Table 5.10 reads its throughput at intermediate values of p_{95} rather than at measured ones. Only four of the thirteen comparisons land within 15% of a measured random run; the rest lean on the estimate, and recomputing them against the nearest measured run instead moves several by more than a tenth and reverses the sign of one. No conclusion is drawn from those rows. The interval $p \in (0, 0.05)$ is unsampled and contains the steepest part of the trade-off; a finer grid there, together with settings between $p = 0.10$ and $p = 0.25$ to bracket the tight-budget adaptive cells and settings chosen to reach the dirty end at high R , would turn most of these readings into measurements and close the six cells where the two arms do not overlap at all.
- **The workload sweeps vary one axis at a time.** X4 and X5 establish that the budget holds across semantic overlap and across resources per transaction, but each holds the other at its baseline. The corner where both are extreme is unsampled, and X4's saturation result at $\sigma = 1.00$, $R = 1$ suggests it is the corner most likely to behave differently.
- **$L = 400$ leaves the regime under test.** At that setting the 30 transactions held in flight demand more resources between them than the system has locks, so lock conflict rather than the oracle limits admission and the abort rate reaches 0.65. The block is a stress anchor, not a scaling point, and is excluded from the invariance claim of Section 5.7.1.

- **The ablation cannot be re-run against current code.** The `alpha_sweep` results behind Section 5.8 are archived under `experiments-old-ewma-0.2/` and the tables can be regenerated from them, but the parameter they sweep no longer exists in the simulator, so the runs themselves cannot be reproduced without reverting the removal. The confirmation in Section 5.8.5 is reproducible against current code; the ablation grid is reproducible only from the archive.

5.11. The earlier filtered runs

Two generations of results exist for this system, and the artefact archive contains both, so it is worth saying at the outset which is which and why the older set was superseded.

Every number in this chapter, unless the text says otherwise, comes from the oracle described in Section 4.6.3, which compares its thresholds against the dirty fraction read at the instant of the decision. An earlier version instead compared them against an exponentially weighted moving average of that fraction,

$$\hat{d}_k = \alpha d_k + (1 - \alpha) \hat{d}_{k-1}, \quad (5.2)$$

where $\alpha \in [0, 1]$ is the smoothing factor and k indexes successive samples; larger α tracks the signal more closely and filters less. The default was $\alpha = 0.2$. One property of the filter matters for what follows: its time constant is expressed in *samples*, whereas the quantity it estimates evolves in *seconds*, and because the oracle samples once per admission rather than once per unit of simulated time (Section 4.6.5), the rate relating the two depends on load.

X1, X2 and X3 were run under both, on three seeds under the filtered oracle and five under the current one; those older files are archived under `experiments-old-ewma-0.2/`.

The filter was introduced as an anti-flap measure, and the reason it was removed is that it did not work as one. Table 5.17 decomposes its effect. It is true that the filter lowered the raw switch count, by 9 to 24%, and read alone that column supports the anti-flap story. The decomposition does not. Switches fall for two quite different reasons — because the system enters ACID less often, or because each visit lasts longer — and only the first is stabilisation.

The BASE dwell columns are the ones that answer the question. At $R = 5, 10, 15$ the filter changes how long the system stays out of ACID by between -7% and $+7\%$ — that is, not at all, to within the seed dispersion. The rate at which the system decides it needs ACID is set by the entry threshold and the workload, and the filter does not touch it. What the filter changes is the other term: each excursion runs 35 to 95% longer, because the smoothed estimate is still above r well after the dirty set has drained below it. The fall in switch count is arithmetic — fewer, longer visits in a fixed run length — and not stability.

The cost of buying that arithmetic is paid in the mode the system is stuck in: ACID time share rises by 10% at $R = 1$ and 77% at $R = 15$ relative to the unfiltered oracle. The filter did not damp an oscillation; it added lag on the exit edge, and the reduced switch count was a symptom of the lag rather than a benefit. Section 5.8 reports the full sweep of α behind this, and Section 5.8.5 the effect of the removal on X1 and X2.

Table 5.17.: What the smoothing filter actually did. *Entries* counts BASE $\rightarrow$ ACID transitions and *BASE dwell* is mean simulated time in BASE per entry, that is, how long the system stayed out of ACID between excursions. $\alpha = 0.2$ is the earlier default, $\alpha = 1.0$ the same code path with the filter disabled, so the two columns differ in nothing else.

<i>R</i>	Entries		BASE dwell per entry (s)			Excursion length (s)		
	$\alpha=0.2$	$\alpha=1.0$	$\alpha=0.2$	$\alpha=1.0$	change	$\alpha=0.2$	$\alpha=1.0$	change
1	33	40	206	264	−22%	1330	987	+35%
5	89	117	227	213	+7%	336	217	+55%
10	93	113	302	314	−4%	236	130	+82%
15	73	81	481	517	−7%	201	103	+95%

Source. experiments-old-ewma-0.2/alpha_sweep/alpha_sweep_seed{0,1,2}.csv, $c = 0.10$, mean over 3 seeds.

“Change” is the filtered value relative to the unfiltered one. BASE dwell is $(1 - \phi_{\text{time}}) \times \text{duration}$ divided by `entry_dirty_live_count`.

6. Conclusion and Future Work

6.1. Summary

6.1.1. RQ1: making the hybrid mode well defined

ACID's lock table and BASE's dirty set are independent mechanisms that never coexist in a pure run. Selecting the mode per transaction allows a resource to be locked and dirty at the same time, which is contradictory. The locked-XOR-dirty arbiter resolves this by making ACID wait on dirty and BASE yield on locked. Because the only cross-mode wait edge points one way, no cross-mode wait cycle can form, and the arbiter introduces no deadlock that pure ACID did not already admit. The resolution is not free; its cost is quantified under RQ2.

6.1.2. RQ2: performance and tunability

The hybrid mode offers a configurable inconsistency budget and charges for it.

The adaptive oracle engages only inside an interior band of reconciler provisioning, $1 \leq R \leq 15$ at the baseline configuration, and is inert outside it; at $R = 0$ and $R \geq 25$ every statistic is identical to pure BASE in every seed. Inside that band, throughput does not interpolate between the two pure modes but falls below a time-weighted mixture of them by a factor of 2.8 to 12.3, with the gap widening as reconciliation slows.

The critical dirty fraction c nonetheless behaves as a configurable bound on the tail. Across c from 2% to 30% and R from 1 to 15, measured p_{95} never departs from configured c by more than 0.74 percentage points, with a median seed-to-seed coefficient of variation of 0.25%. X4 and X5 extend the same result across two workload axes: it spans 1.04 pp across semantic overlap from 0.20 to 0.50 and 0.78 pp as the resources per transaction grow by a factor of 20, in both cases straddling the configured value. It stops applying in two predictable and detectable ways: once c exceeds the BASE steady-state dirtiness at that R , and once reconciliation capacity falls below the dirtying rate the workload imposes, at which point the oracle saturates in ACID and misses the budget regardless of configuration.

6.1.3. RQ3: is the feedback necessary?

The answer depends on what is being asked for, and the two halves point in opposite directions.

On the trade-off itself, feedback is not necessary — or at least, no advantage for it can be established here. Compared at the same measured inconsistency, the two oracles turn in the same throughput wherever an adaptive run lands close enough to a measured random run for the comparison to be nearly direct. Where the data hints at a difference, the random sweep is

too coarse to support the hint, and recomputing those rows against the nearest measured run moves several of them substantially and reverses one. Reading the system's own metrics buys no throughput advantage over a coin flip that can be demonstrated, and costs none either.

That result should be read with the workload in mind. Every run in this thesis is executed under a stationary load, and an adaptive controller has nothing to adapt to when the operating point never moves; the experiment was always going to be a poor place for feedback to show its value. Running the same comparison under a workload that changes while the run is in progress is the single most valuable extension of this work (Section 6.3), and until it is done, RQ3 is answered only for the stationary case.

The other half is not a throughput result at all. Each oracle holds fixed the quantity it reads and lets the other float: the random probability realises its configured ACID share to within 9% across the whole reconciler range while its measured p_{95} moves by up to a factor of 43.9, and the adaptive threshold is the mirror image, holding p_{95} within 0.74 percentage points of the configured c while its realised ACID share varies by a factor of 4.4. Only one of those two quantities is the one an operator wants to bound. Holding a 10% tail takes $p = 0.15$ at $R = 1$ and about 0.06 at $R = 15$, a setting that has to be found again for every deployment; $c = 0.10$ holds it everywhere.

The adaptive oracle therefore does not make the ACID/BASE trade-off cheaper. It makes it addressable, and it does so for free.

6.2. Lessons learned

- A HYBRID run with zero mode switches *is* pure BASE with a different label. Engagement must be verified before a result is interpreted: five of the 24 cells in X2 are degenerate in exactly this way, and thirteen of the 28 in X4.
- A configured threshold bounds the estimate, not the underlying quantity. The two coincide only when the system is slow relative to the sampler, and when they diverge, the direction of the error depends on which mode the condition is evaluated in.
- Composing two correct mechanisms does not yield a mechanism whose cost is the weighted average of theirs.
- A metric with three plausible definitions needs all three named before any of them is reported. Choosing one and describing another produced a headline number that was wrong by a factor of six, and the error survived several readings because both quantities are fractions and both were plausible in size.
- An analytical model can be qualitatively right and mechanistically wrong. The blocking probability derived in Section 4.4.3 is close to one across most of the operating range, which the measurements confirm — and that is precisely why it explains nothing: a saturated variable has no variation left with which to explain a response.

- Comparing a controller against a baseline without feedback requires deciding which variable is held constant. If that variable is an input to one arm and an output of the other, the comparison is not well posed, however carefully it is measured. The way out is to match on a quantity that is an output of both arms and to interpolate the baseline onto it, rather than to pair up whichever settings happen to land near each other: matching to the nearest sampled cell discarded most of the data and left a residual mismatch as large as the effect being measured.

6.3. Future work

1. **Non-stationary workloads** (terminal-based arrivals, phase-changing overlap). This is the setting in which adaptive control would be expected to show the advantage that RQ3 could not find at fixed provisioning under stationary load, and it is the most valuable single extension.
2. **A finer grid of random probabilities.** The random arm is sampled at five values of p , and most of the X3 comparisons fall too far from any of them to be conclusive (Section 5.5.2). Sampling between $p = 0.10$ and $p = 0.25$, and below $p = 0.05$ where the trade-off is steepest, would turn those readings into measurements.
3. **Tick-driven signal sampling**, to decouple the filter's time constant from the load it measures. Because admissions track completions under EQUILIBRIUM, the oracle's sampling rate is approximately the throughput its own decisions suppress, and the measured interval varies by a factor of 60 across the sweep.
4. **Within-excursion sampling analysis.** The undershoot of the recovery threshold at exit is measured, but the mechanism that makes it grow with R is not established, and the aggregate sample counts rule out the obvious explanation. The tick timelines record per-tick state and would settle it.
5. **A time-based oracle clock.** The oracle samples once per admission, so its sampling rate collapses with the throughput it causes. Sampling on a fixed interval of simulated time would decouple the two and is the single change most likely to close the remaining exit slack.
6. **Absolute dirty thresholds** alongside fractional ones, for small systems.
7. **Predictive switching**, on the dirty *trend* rather than the level.
8. **Lock-granularity aliasing** in the BASE yield check.
9. **Real-system validation** against the Vitruvius case-study parameters under HYBRID.
10. **Transaction-size interaction.** The blocking mechanism makes the maximum transaction size L a hidden co-determinant of the mixing cost. Sweeping L at fixed c would test the mechanism directly: since block duration rather than incidence carries the cost, L

should move the number of dirty resources waited on, and therefore the block duration, roughly linearly.

11. **Closing the random-split frontier at small p .** The interval $p \in (0, 0.05)$ is unsampled and contains the steepest part of the trade-off.
12. **Reporting the arbiter invariant empirically.** The locked-XOR-dirty invariant and the deadlock-freedom argument (Sections 4.4.1 and 4.4.2) are established by construction and on paper; no measurement of them is reported. The evidence already exists in the run data and would need no new runs. Both arbiter paths are counted per run as `acid_txn_blocked_on_dirty_count` and `base_txn_yielded_on_locked_count`, and both are exactly zero in every pure ACID and pure BASE run and at the degenerate adaptive points $R = 0$ and $R = 25$, while in the engaged band they number in the hundreds and tens of thousands respectively. Reporting them would show that the mixed state was reached and resolved constantly rather than merely never observed, which is what makes “the invariant was never violated” a claim with content. It would also confirm the degeneracy of $R = 0$ and $R = 25$ through a mechanism independent of the switch count. Two cautions apply to the wording: the invariant is enforced by construction at the two write paths, not checked by a runtime assertion — the Java `assert` statements in the codebase were disabled in every experiment run, as no harness passes `-ea` — so the claim to make is that the hazardous state cannot be constructed, evidenced by how often the enforcing branch was taken. The natural home is Section 4.4, after the deadlock argument, rather than the experiments chapter, since neither property varies with R or c .

A. Additional material

A.1. Full parameter table

The complete configuration of the simulator with its baseline value for every parameter, placed next to the parameter table of the reference paper so that the two can be compared line by line.

A.2. Per-seed result tables

Chapter 5 reports means over five seeds. This section gives the per-seed rows for all five sweeps, from which the seed-to-seed variation behind those means can be checked.

A.3. Full grids

The complete blocking-counter grid behind Table 5.7, the complete smoothing-factor grid behind Table 5.13, and the full adaptive-versus-random cell grid behind Table 5.10, including the twelve engaged cells that no random cell matches to within the tolerance.

A.4. Implementation defects found and fixed

The defects surfaced while hardening the received code, each with its symptom, its fix, and the metric it would otherwise have distorted.

A.5. Supplemental material and AI usage disclosure

The repository containing the simulator, the experiment runner scripts and the exported result files, followed by a disclosure of which AI tools were used and how, separated into conceptualising, developing code, writing and proofreading.

List of Figures

1.1. Motivating example: an update to one model propagates to two others through consistency preservation rules, either inside the transaction or after it commits.	1
1.2. Overview of the approach: mode assignment at admission, the two execution paths, and the arbiter between them.	4
4.1. The mixing hazard: with modes chosen per transaction, a resource can be held by an ACID lock and be in the dirty set at the same time.	13
4.2. Dispatcher architecture: one admission path, two dispatchers, shared processors, lock table, dirty set and reconciler pool.	14
4.3. The adaptive oracle as a two-state machine, with the entry and exit conditions on the edges.	15
5.1. O2: the adaptive throughput valley.	25
5.2. O2: the uncontrolled dirty fraction against c , which decides whether the oracle fires at all. Dotted lines are p_{95}	25
5.3. O3: the ACID time share and the switch count peak at different R	26
5.4. O4: the random oracle as a negative control, flat where the adaptive oracle has a valley.	27
5.5. How far the measured p_{95} dirty fraction lands from the configured threshold c , one line per reconciler count. Zero is a run that hit its setting exactly. The band marks the largest deviation seen in any single run, ± 0.74 pp. Degenerate cells are left out.	30
5.6. O7: throughput against the configured threshold c , one line per reconciler count. Degenerate cells are left out, which is why the $R = 15$ line stops at $c = 0.15$	32
5.7. Measured p_{95} dirty fraction against reconciler count, one line per knob setting, on a shared logarithmic scale. Each oracle holds fixed the quantity it reads: the adaptive lines are flat because the threshold is a target, and the random lines fan out because the probability is not.	36
5.8. What the thresholds were set to, and where the oracle actually switched, at $c = 0.10$ and $r = 0.05$. Entry lands on c at every reconciler count; exit falls short of r by a distance that grows with reconciliation capacity. The short bars are the p_{95} exit, the dirtiest observed. Means over five seeds, the data of Table 5.16.	53

List of Tables

2.1. Notation used throughout this thesis.	8
3.1. Differentiation from related work.	11
4.1. Received work against own contribution.	12
5.1. Baseline configuration.	18
5.2. X0: arbiter activity across all runs in this chapter, by execution mode.	21
5.3. X1: results by reconciler count R and configuration.	23
5.4. HYBRID_ADAPTIVE throughput as measured, against the weighted average of Equation 5.1.	28
5.5. X2: measured p_{95} dirty fraction against the configured threshold.	29
5.6. X2: mean throughput and mode-switch count against the configured threshold.	32
5.7. X2: what an ACID transaction costs, and how much of the run is ACID.	33
5.8. X3: results by ACID probability p and reconciler count R . Columns are those of Table 5.3: ϕ_{time} is the share of the run spent in ACID mode, ϕ_{comp} the share of completed transactions that ran under ACID, and <i>Switches</i> how often the oracle changed the mode.	36
5.9. Where each adaptive run falls among the random runs at the same reconciler count. <i>Between</i> gives the dirtiness of the two random runs on either side; the full set of random runs is the p_{95} column of Table 5.8. <i>To nearest</i> is how far the adaptive run sits from the closer of those two, as a ratio — 1.00 means it landed on top of a measured run. A run with no random run on one side cannot be compared and is dropped rather than extrapolated.	39
5.10. Throughput of the two oracles at the same measured inconsistency, for the thirteen adaptive runs that fall between two random runs. <i>Ratio</i> is adaptive divided by random. Rows are ordered by <i>To nearest</i> (Table 5.9), so the comparisons that are closest to a straight measurement come first; the rule marks where the adaptive run stops sitting within 15% of a measured one.	40
5.11. X4: dirty-fraction percentiles, oracle behaviour, throughput and abort rate against the semantic overlap degree σ , by reconciler count R . “deg.” marks a cell in which the oracle never switches, so the row is pure BASE under a different label. Abort rate is <i>aborted_total</i> over <i>admitted-plus-aborted</i>	42
5.12. X5: what moves and what does not as the resources per transaction L grow, by reconciler count R	45

5.13. Ablation: dirty-fraction and oracle-behaviour statistics by α and R . The $\alpha = 0$ row is the degenerate case, included as a convergence check against pure BASE. Intermediate values of α are omitted here; the full grid is in the appendix. . . .	48
5.14. Ablation: the effect of removing the smoothing. $\alpha = 0.2$ is the earlier default, $\alpha = 1.0$ the unfiltered oracle. $\bar{d}_{\text{exit}}$ is the mean live dirty fraction at the instant the oracle returns to BASE — the effective recovery threshold of Section 5.9 — against a configured $r = 5\%$	49
5.15. Effect of removing the smoothing, measured by re-running X1. “before” is the filtered oracle at the earlier default $\alpha = 0.2$, three seeds; “after” is the filter-free oracle, five seeds. The guarantee (p_{95} against $c = 10\%$) is unchanged; the cost of providing it falls.	51
5.16. Where the oracle actually switches, how long an ACID transaction waits, and how often the oracle looks. Measured under HYBRID_ADAPTIVE at $c = 0.10$, $r = 0.05$. Dirty fractions are percentages of the 5000 resources; the <i>miss</i> columns are the distance from the threshold that fired, in percentage points, positive meaning dirtier than configured.	52
5.17. What the smoothing filter actually did. <i>Entries</i> counts BASE $\rightarrow$ ACID transitions and <i>BASE dwell</i> is mean simulated time in BASE per entry, that is, how long the system stayed out of ACID between excursions. $\alpha = 0.2$ is the earlier default, $\alpha = 1.0$ the same code path with the filter disabled, so the two columns differ in nothing else.	57

Bibliography

- [1] G. Weikum and G. Vossen. *Transactional Information Systems: Theory, Algorithms, and the Practice of Concurrency Control and Recovery*. Elsevier, 2001.
- [2] M. Hsu and B. Zhang. “Performance evaluation of cautious waiting”. In: *ACM Transactions on Database Systems* 17.3 (1992), pp. 477–512. doi: 10.1145/132271.132275.
- [3] D. Pritchett. “BASE: An ACID Alternative”. In: *ACM Queue* 6.3 (2008), pp. 48–55.
- [4] W. Vogels. “Eventually consistent”. In: *Communications of the ACM* 52.1 (2009), pp. 40–44.
- [5] H. Klare, M. E. Kramer, M. Langhammer, D. Werle, E. Burger, and R. Reussner. “Enabling consistency in view-based system development — the Vitruvius approach”. In: *Journal of Systems and Software* 171 (2021), p. 110815.
- [6] Z. Zacouris and M. Acosta. “Simulating a Transactional Server for Multi-Model Systems”. In: *Proceedings of the VLDB Endowment* 18.12 (2025), pp. 5395–5398. doi: 10.14778/3750601.3750680.
- [7] Z. Zacouris and M. Acosta. “A Transaction Simulator for Updates in Consistency-Aware Multi-Model Systems”. Under submission. 2026.
- [8] C. Xie, C. Su, M. Kapritsos, Y. Wang, N. Yaghmazadeh, L. Alvisi, and P. Mahajan. “Salt: Combining ACID and BASE in a Distributed Database”. In: *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. USENIX Association, 2014.
- [9] C. Li, D. Porto, A. Clement, J. Gehrke, N. Preguiça, and R. Rodrigues. “Making Geo-Replicated Systems Fast as Possible, Consistent when Necessary”. In: *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)*. 2012.
- [10] T. Kraska, M. Hentschel, G. Alonso, and D. Kossmann. “Consistency Rationing in the Cloud: Pay only when it matters”. In: *Proceedings of the VLDB Endowment* 2.1 (2009), pp. 253–264.
- [11] H.-E. Chihoub, S. Ibrahim, G. Antoniu, and M. S. Pérez. “Harmony: Towards Automated Self-Adaptive Consistency in Cloud Storage”. In: *IEEE International Conference on Cluster Computing (CLUSTER)*. 2012.
- [12] H. Yu and A. Vahdat. “Design and Evaluation of a Continuous Consistency Model for Replicated Services”. In: *4th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2000)*. 2000, pp. 305–318.
- [13] D. R. Ries and M. R. Stonebraker. “Locking granularity revisited”. In: *ACM Transactions on Database Systems* 4.2 (1979), pp. 210–227. doi: 10.1145/320071.320078.

- [14] S. Dandamudi and S.-L. Au. "Locking granularity in multiprocessor database systems". In: *Proceedings of the Seventh International Conference on Data Engineering*. IEEE, 1991, pp. 268–269.